

FINAL PROJECT 2 REPORT

Live Webcam Program for Heart Rate Estimation using Remote Photoplethysmography (rPPG) Detection based on Forehead and Cheek Region of Interest (ROI)



Name : Jeremia Christ Immanuel Manalu
NRP : 5023231017
Course : Biomedical Image Processing
Class : A
Lecturer : Nada Fitrieyatul Hikmah, S.T., M.T.
Department : Biomedical Engineering

**FACULTY OF INTELLIGENT ELECTRICAL AND INFORMATICS
TECHNOLOGY**

INSTITUT TEKNOLOGI SEPULUH NOPEMBER

2026

CHAPTER I. FUNDAMENTAL THEORY

1.1. Input Webcam

In the context of remote Photoplethysmography (rPPG), a standard webcam serves as a non-contact optical sensor that captures ambient light reflected off the human skin. Unlike traditional contact-based PPG sensors (such as pulse oximeters) that utilize dedicated infrared or red LEDs to measure blood volume variations, a webcam relies exclusively on environmental illumination and a standard RGB (Red, Green, Blue) CMOS or CCD sensor. As the heart pumps blood, the volume of microvascular beds in the facial tissue subtly changes, which in turn alters the absorption and reflection of ambient light. The webcam continuously records these minute photometric variations, providing the foundational visual data required to extract the cardiovascular pulse signal remotely.

The technical specifications of the webcam, particularly its spatial resolution and temporal sampling rate, critically govern the quality of the resulting rPPG signal. The sampling rate, measured in Frames Per Second (FPS), must satisfy the Nyquist-Shannon sampling theorem to accurately reconstruct the heart rate frequency. Given that the typical human heart rate ranges from 45 to 150 Beats Per Minute (BPM), which equates to 0.75 Hz to 2.5 Hz, the camera must capture at least double the maximum frequency (i.e., > 5 FPS). In practice, a standard operating frame rate of 30 FPS is highly sufficient, providing a high-density temporal resolution that minimizes aliasing and captures the fine waveform morphological details required for precise peak detection.

However, utilizing a commercial webcam introduces inherent hardware and environmental challenges. Webcams frequently employ automatic exposure (AE) and automatic white balance (AWB) algorithms to dynamically adjust to changing ambient light. While beneficial for human viewing, these automatic adjustments can introduce artificial intensity shifts in the video feed that mimic or obscure the subtle blood volume pulse signals. Consequently, the raw input from the webcam is inherently noisy, necessitating robust downstream computer vision and signal processing techniques to isolate the true physiological parameters from hardware-induced photometric artifacts.

1.2. Frame Acquisition

Frame acquisition is the continuous, sequential process of retrieving individual image frames from the webcam's video stream into the computer's operational memory. During this stage, the continuous optical analog signal is digitized and quantized into a discrete sequence of images. In OpenCV, frames are typically fetched in a BGR (Blue, Green, Red) color space by default, which is immediately accessible as a three-dimensional multidimensional array (matrix) composed of 8-bit unsigned integers representing pixel intensities ranging from 0 to 255.

Temporal stability during frame acquisition is paramount for the integrity of the rPPG pipeline. For accurate frequency-domain analysis, such as the Fast Fourier Transform (FFT) used later in the system, it is mathematically assumed that the time interval (Δt) between consecutive frames is strictly constant. Fluctuations in the acquisition process, often caused by CPU/GPU resource contention or I/O bottlenecks, result in temporal jitter. To mitigate the impact of this jitter, modern rPPG systems accurately timestamp each acquired frame. Tracking the precise acquisition time allows the system to compute the true operational FPS and apply temporal interpolations or adaptive filtering if severe frame-dropping occurs.

Furthermore, the acquired frame serves as the raw spatial canvas for the subsequent detection algorithms. At this stage, no dimensional reduction or color manipulation is performed; the integrity of the raw pixel data is preserved. The output of the frame acquisition module is a high-fidelity chronological sequence of digital matrices, $F(t_n)$, where t_n represents the discrete timestamp of the n -th frame, ready to be processed by facial detection frameworks.

1.3. Image Enhancement

To extract the blood volume pulse from a subject, the system must first locate the skin regions within the acquired frame, an operation accomplished through facial frame detection. This project utilizes the Viola-Jones object detection framework, specifically employing Haar Cascade Classifiers. Haar-like features are digital image features used to recognize objects by calculating the difference in pixel intensities between adjacent rectangular regions. For instance, the algorithm leverages the natural physiological contrast of the human face, recognizing that the eye region is generally darker than the upper cheeks and forehead. By evaluating thousands of these rectangular features, the classifier can rapidly distinguish a human face from the background.

The computational efficiency of the Haar Cascade algorithm is driven by two core mathematical concepts: the Integral Image and the AdaBoost learning algorithm. The Integral Image allows the sum of pixels within any rectangular area of the image to be calculated in constant time, $O(1)$, drastically accelerating the feature evaluation process. Concurrently, AdaBoost constructs a "strong" classifier by combining multiple "weak" classifiers, effectively discarding non-face regions at the earliest possible stages of processing or the cascade mechanism. This ensures that the detection process maintains real-time performance, which is a mandatory requirement for continuous live rPPG monitoring.

In the implementation of this system, specific detection parameters such as `scaleFactor` and `minNeighbors` are finely tuned. The `scaleFactor` dictates how much the image size is reduced at each image scale to detect faces of varying distances, while `minNeighbors` specifies how many neighboring rectangles must be positively identified to retain a bounding box, acting as a threshold against false positives. Once the frontal face is detected, the algorithm further detects the eye regions within the upper proportion of the face. This multi-level detection creates a robust geometric reference frame, ensuring that the subsequent skin tracking remains tightly anchored to the subject's anatomy regardless of minor translational head movements.

1.4. Performance Evaluation

Following successful facial detection, the system must isolate specific regions of the skin that are most indicative of blood volume variations, known as Regions of Interest (ROI). Not all areas of the face are suitable for rPPG, regions containing hair, eyebrows, or excessive topological variations introduce severe optical noise. Consequently, the forehead and the upper cheeks are universally selected as the optimal ROIs. These areas provide relatively flat anatomical surfaces with a high concentration of subcutaneous microvasculature (capillaries), yielding the strongest and most consistent pulsatile photometric signals.

The ROI selection is executed dynamically by calculating proportional coordinates relative to the primary facial bounding box detected by the Haar Cascade. Instead of using static pixel coordinates, which would fail if the subject moves towards or away from the

camera, the system defines the forehead and cheeks using fractional multipliers (e.g., targeting the top 20% of the bounding box width for the forehead). This dynamic spatial mapping ensures that the selected regions scale automatically and consistently encompass the correct facial tissue across varying distances and resolutions, maintaining signal continuity.

To maximize the robustness of the heart rate estimation, a multi-ROI strategy is implemented. The system independently extracts data from the Forehead (Mode A), the averaged Left and Right Cheeks (Mode B), and a Combined average of all distinct areas (Mode C). This redundancy is crucial in real-world scenarios where localized artifacts, such as a shadow falling across one side of the face or minor head rotations, might corrupt a single region. By maintaining parallel data streams from multiple ROIs, the system can cross-validate the signals, ensuring high-fidelity outputs even under heterogeneous illumination conditions.

1.5. Image Restoration

Signal extraction is the pivotal bridge between the spatial domain (images) and the temporal domain (time-series signals). Once the ROIs are geometrically defined, the system extracts the raw color information by performing spatial averaging across all pixels within each designated boundary. For each discrete frame at time t , and for each color channel $c \in \{R, G, B\}$, the spatially averaged signal $y_c(t)$ is mathematically defined as:

$$y_c(t) = \frac{1}{W \cdot H} \sum_{i=1}^W \sum_{j=1}^H I_c(i, j, t)$$

where W and H represent the width and height of the ROI, and $I_c(i, j, t)$ is the intensity of the pixel at coordinates (i, j) .

The primary purpose of spatial averaging is dramatic noise reduction. A webcam's image sensor is inherently afflicted by thermal and quantization noise (shot noise), which operates as independent, zero-mean Gaussian random variables across adjacent pixels. By summing and averaging thousands of pixels within the ROI, the spatial noise effectively cancels itself out. This spatial aggregation significantly elevates the Signal-to-Noise Ratio (SNR) of the raw color trace, ensuring that the microscopic color variations driven by the cardiac cycle, which are often invisible to the naked human eye, become mathematically dominant in the resulting data point.

Finally, the spatially averaged RGB triplets extracted from each consecutive frame are concatenated chronologically into a continuous buffer structure, such as a rolling window queue spanning a specified temporal length (e.g., 10 seconds). This process generates the raw multichannel time-series matrix $S(t) = [R(t), G(t), B(t)]^T$. This structured array captures the continuous fluctuations of skin color over time and serves as the fundamental physiological dataset. The RGB time series is now ready to be fed into advanced digital signal processing filters and chrominance-based projection methods to isolate the pure rPPG waveform.

1.6. CHROM Method

The Chrominance-based (CHROM) method is a highly robust rPPG algorithm designed to extract the blood volume pulse (BVP) while actively suppressing motion artifacts and specular reflection. Proposed by De Haan and Jeanne, the CHROM method addresses a fundamental flaw in single-channel extraction methods: the human skin acts

as a dichromatic surface. Light reflected from the skin consists of diffuse reflection, which contains the physiological pulsatile information from subcutaneous blood vessels and specular reflection which bounces off the skin's surface like a mirror and contains purely the spectrum of the light source. When a subject moves, the specular reflection shifts drastically, introducing severe baseline wandering and noise into the RGB signals.

To mitigate this, the CHROM method mathematically projects the standardized RGB signals into two orthogonal chrominance vectors. First, the raw spatial averages of the Red, Green, and Blue channels are normalized by dividing them by their respective temporal means (R_n, G_n, B_n), which balances the white-balance disparities of the camera. Then, these normalized signals are mapped into two chrominance signals: $X_s = 3R_n - 2G_n$ and $Y_s = 1.5R_n + G_n - 1.5B_n$. These specific color-difference equations are constructed based on a standardized skin-color profile under white illumination, purposefully designed to be independent of the specular reflection component.

The final rPPG waveform (S) is computed by subtracting the second chrominance signal from the first, scaled by an adaptive ratio parameter known as α . Mathematically, this is expressed as $S = X_s - \alpha Y_s$, where α is the ratio of the standard deviations of the two signals: $\alpha = \frac{\sigma(X_s)}{\sigma(Y_s)}$. This adaptive α acts as an automatic noise-cancellation coefficient. Because specular reflection noise affects all color channels identically, scaling and subtracting Y_s from X_s using this variance ratio eliminates the specular artifacts, leaving behind a highly purified diffuse reflection signal that accurately represents the cardiovascular pulse.

1.7. Preprocessing (Filtering)

In digital signal processing, preprocessing acts as an essential noise removal stage, conceptually similar to spatial image restoration but applied in the temporal (1D) domain. The raw rPPG signal extracted from the facial ROI is heavily contaminated by out-of-band noise, such as slow baseline drifts caused by respiration typically < 0.5 Hz, and high-frequency fluctuations caused by camera quantization noise or artificial lighting flicker typically > 3.0 Hz. To isolate the physiological heart rate, an Infinite Impulse Response (IIR) Biquad Bandpass Filter (BPF) is utilized. The filter is mathematically tuned to allow only frequencies between 0.75 Hz and 2.5 Hz to pass, tightly corresponding to a valid human heart rate range of 45 to 150 BPM.

To maintain the morphological integrity of the waveform, which is critical for accurate peak detection, the temporal filtering must not shift the phase of the signal. Traditional IIR filters introduce non-linear phase delays, causing the systolic peaks of the pulse wave to be displaced in time. To counteract this, a zero-phase filtering technique, often referred to as forward-backward filtering, is implemented. The signal is first filtered in the forward direction, reversed, filtered again, and then reversed back to its original orientation. This process mathematically guarantees a zero-phase shift, ensuring that the temporal coordinates of the blood volume pulses remain anatomically precise.

Further signal refinement is achieved through Savitzky-Golay (SG) smoothing. While standard moving average filters (box filters) can suppress high-frequency noise, they suffer from a major drawback: they significantly flatten and distort the sharp peaks of the waveform. The Savitzky-Golay filter elegantly solves this by performing a local polynomial regression on a sliding window of the data. For a given window size and polynomial order, the filter calculates a set of convolution coefficients using a pseudo-inverse Vandermonde matrix. Convolution of the signal with these coefficients smooths the

data while preserving the height and width of local maxima and minima, producing a high-fidelity continuous wave perfectly suited for subsequent frequency analysis.

1.8. Remote Photoplethysmography (rPPG)

Remote Photoplethysmography (rPPG) is an innovative, non-contact physiological measurement technique that translates the optical properties of human skin into cardiovascular metrics. Traditional Photoplethysmography (PPG), widely used in clinical pulse oximeters, requires a dedicated active light source (LED) and a photodiode clamped directly to the skin to measure the volumetric changes of arterial blood. In contrast, rPPG utilizes ambient environmental light as the illumination source and a standard digital camera as the receptor. As the heart undergoes systole and diastole, the periodic variation of blood volume in the facial microvascular bed dynamically alters the skin's light absorption and reflection spectra.

The physiological basis of rPPG relies heavily on the optical absorption characteristics of hemoglobin. Oxygenated hemoglobin (HbO_2) exhibits distinct light absorption peaks, particularly in the green spectrum, around 500-600 nm. Consequently, the green channel of an RGB camera typically yields the strongest pulsatile signal amplitude. During a cardiac systole, blood volume increases in the facial tissue, leading to a higher absorption of green light and a corresponding micro-drop in the pixel intensity captured by the camera. Conversely, during diastole, the blood volume decreases, light absorption drops, and pixel intensity slightly recovers.

Despite its tremendous potential for non-invasive continuous monitoring, rPPG faces substantial real-world challenges. The desired blood volume signal is microscopic, often constituting less than 1% of the total pixel intensity variations. The remaining 99% consists of severe interference from rigid head movements, facial expressions, and dynamic ambient illumination changes. Therefore, a successful rPPG system cannot rely on a single algorithm; it mandates a robust, multi-stage computational pipeline. It requires precise spatial tracking to lock onto the skin, sophisticated color-space transformations like the CHROM method to cancel surface reflections, and rigorous temporal filtering to extract the deeply buried physiological truth.

1.9. Frequency Analysis (FFT) and Peak Detection

Once the rPPG signal is heavily filtered and smoothed, the system must translate the continuous time-domain waveform into a quantifiable heart rate metric. The primary mathematical approach for this is Frequency Analysis via the Discrete Fourier Transform (DFT), optimized computationally as the Fast Fourier Transform (FFT). The FFT algorithm decomposes the complex, periodic 1D signal into its constituent sinusoidal frequencies. Before executing the FFT, the signal is multiplied by a Hanning window function: $w[n] = 0.5 \left(1 - \cos\left(\frac{2\pi n}{N-1}\right) \right)$. This windowing tapers the edges of the signal array to zero, effectively suppressing spectral leakage that would otherwise smear the frequency peaks across adjacent bins.

By analyzing the resulting power spectrum, the system can identify the dominant periodic component. The frequency bin containing the maximum magnitude or highest peak within the predefined physiological bandwidth (0.75 Hz to 2.5 Hz) is designated as the fundamental heart rate frequency. The FFT provides a highly robust estimation of the average heart rate over the duration of the data buffer, as it is inherently resilient to transient, high-frequency noise spikes that might still linger in the time-domain signal.

Furthermore, the ratio of the peak magnitude to the total spectral energy provides a vital "Confidence Score," indicating the reliability of the current measurement.

In parallel to frequency analysis, time-domain Peak Detection is employed to analyze the beat-to-beat variations. Utilizing local maxima finding algorithms (e.g., `scipy.signal.find_peaks`), the system identifies the exact temporal coordinates of each systolic peak in the rPPG waveform. To prevent false positives caused by minor signal ripples, the algorithm enforces a minimum prominence threshold, the vertical distance between the peak and its lowest contour line and a minimal temporal distance between consecutive peaks based on the maximum allowable heart rate. The time difference between adjacent peaks defines the Inter-Beat Interval (IBI), from which the instantaneous heart rate can be inversely calculated.

1.10. Heart Rate (BPM) Calculation and Kalman Filtering

The final derivation of the Heart Rate, measured in Beats Per Minute (BPM), is a direct scalar conversion from the analyses. From the frequency domain, the dominant peak frequency (f_{peak} in Hz) is multiplied by 60 ($BPM = f_{peak} \times 60$). Alternatively, from the time domain, the average Inter-Beat Interval (IBI_{mean} in seconds) is used to calculate the rate as $BPM = \frac{60}{IBI_{mean}}$. While these direct calculations are mathematically sound, the raw BPM outputs in live environments often suffer from erratic fluctuations due to sudden user movements or temporary camera exposure adjustments.

To ensure medical-grade stability and smooth out transient outliers, an Interquartile Range (IQR) weighted median filter is initially applied to a short history buffer of the raw BPM values. This statistical approach aggressively rejects extreme outliers. However, median filtering alone lacks a predictive understanding of physiological trends. To solve this, a 2-State Constant-Velocity Kalman Filter is introduced as the final tracking mechanism. The Kalman Filter models the hidden true state of the heart rate as a 2D vector comprising the current BPM and its temporal trend or the rate of change or velocity of the heart rate.

The Kalman Filter operates in a continuous predict-update loop. First, it uses the kinematic state transition matrix to predict the next BPM based on the current trend. Second, it incorporates the new incoming BPM measurement to correct its prediction. A unique feature of this implementation is its adaptive measurement noise covariance (R). The covariance is dynamically scaled inversely to the FFT's confidence score. If the FFT yields a low-confidence reading (e.g., due to the subject covering their face), the Kalman Filter mathematically increases the measurement noise parameter, causing it to ignore the noisy input and instead rely on its internal trend prediction until a clean signal is restored.

CHAPTER II. RESULT AND ANALYSIS

2.1. Problems Statement

→ Create a live webcam program for face detection, eye detection, and HR estimation. The pipeline is as follows:

- A. Webcam
- B. Frame acquisition
- C. Frame detection (HAAR)
- D. ROI selection (forehead/cheek)
- e. Signal extraction (outside of RGB time)
- F. Preprocessing (filtering)
- G. rPPG
- H. Frequency analysis (FFT)/Peak detection
- I. Heart rate (BPM)

2.2. Code Explanation

To construct a robust, real-time, and medical grade rPPG system, a highly modularized software architecture was adopted. By decoupling the monolithic pipeline into distinct, specialized Python modules, the system adheres to the principle of Separation of Concerns (SoC). This sub-chapter meticulously dissects the source code, breaking down the essential classes, functions, and the underlying mathematical logic implemented within each directory of the project.

2.2.1. Core Modules

The `core/` directory serves as the computational brain of the application. It houses six fundamental scripts responsible for transitioning raw optical data from the camera into a clean, physiological heart rate estimation. The execution sequence naturally flows from frame acquisition, spatial detection, region extraction, temporal filtering, chrominance transformation, and finally, frequency analysis.

1. Frame Acquisition (*webcam.py*)

The `WebcamCapture` class acts as the absolute starting point of the pipeline. Its primary objective is to interface with the hardware sensor and retrieve consecutive frames with strict temporal consistency.

```
1. class WebcamCapture:
2.     def __init__(self, camera_index=0, width=640, height=480, fps=30):
3.         self.camera_index = camera_index
4.         self.width = width
5.         self.height = height
6.         self.fps = fps
7.         self.cap = None
8.         self.actual_fps = fps
9.
```



```

10. def open(self):
11.     self.cap = cv2.VideoCapture(self.camera_index)
12.     if not self.cap.isOpened():
13.         print("[WebcamCapture] ERROR: camera {self.camera_index}")
14.         return False
15.
16.     self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, self.width)
17.     self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, self.height)
18.     self.cap.set(cv2.CAP_PROP_FPS, self.fps)
19.
20.     self.actual_fps = self.cap.get(cv2.CAP_PROP_FPS) or self.fps
21.     aw = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH))
22.     ah = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
23.     print("[WebcamCapture] {aw}x{ah} @ {self.actual_fps:.1f} FPS")
24.
25.     return True
26.
27. def read_frame(self):
28.     if self.cap is None or not self.cap.isOpened():
29.         return False, None
30.     return self.cap.read()
31.
32. def release(self):
33.     if self.cap:
34.         self.cap.release()
35.         self.cap = None
36.         print("[WebcamCapture] Released.")
37.
38. def __enter__(self):
39.     self.open()
40.     return self
41.
42. def __exit__(self, *a):
43.     self.release()

```

The *open()* method utilizes the *cv2.VideoCapture* object to establish a direct connection with the local camera index. Crucially, the code explicitly enforces the frame width, frame height, and the Frames Per Second (FPS) using the *cap.set()* directives. Enforcing a constant FPS is mathematically critical for the validity of the subsequent Discrete Fourier Transform (DFT); if the temporal distance (Δt) between sampled frames fluctuates, the frequency bins in the spectral analysis will be heavily distorted.

By attempting to force the hardware to output at a constant 30 FPS, the system ensures that the Nyquist-Shannon sampling criterion is comfortably met. Furthermore, the script reads back the *actual_fps* granted by the hardware. This parameter is globally propagated throughout the pipeline to define the sliding window lengths and the digital filter coefficients, ensuring that all time-series mathematics operate on the true acquisition rate rather than a theoretical assumption.

2. Spatial Localization (*detector.py*)

The *FaceEyeDetector* class is responsible for spatial localization, isolating the subject's skin from the background environment using Viola-Jones Haar Cascades.

```
1. class FaceEyeDetector:
2.     def __init__(self, face_scale=1.1, face_neighbors=5, face_min=(80, 80),
3.                 eye_scale=1.1, eye_neighbors=10, eye_min=(20, 20)):
4.
5.         self.fc = cv2.CascadeClassifier(cv2.data.haarcascades +
6.         'haarcascade_frontalface_default.xml')
7.         self.ec = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_eye.xml')
8.
9.         self.fp = (face_scale, face_neighbors, face_min)
10.        self.ep = (eye_scale, eye_neighbors, eye_min)
11.        print("[FaceEyeDetector] Haar Cascade dimuat.")
12.
13.    def detect(self, frame):
14.        gray = cv2.equalizeHist(cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY))
15.        faces = self.fc.detectMultiScale(gray, *self.fp[:2], minSize=self.fp[2])
16.
17.        res = {'faces':[], 'eyes': {}, 'gray': gray}
18.        if len(faces) == 0:
19.            return res
20.
21.        for idx, (fx, fy, fw, fh) in enumerate(faces):
22.            res['faces'].append((fx, fy, fw, fh))
23.            # Eye ROI only taken from upper part of face (top 60%)
24.            roi_gray = gray[fy:fy + int(fh * 0.6), fx:fx + fw]
25.            eyes = self.ec.detectMultiScale(roi_gray, *self.ep[:2], minSize=self.ep[2])
26.            res['eyes'][idx] = [(fx + ex, fy + ey, ew, eh) for ex, ey, ew, eh in eyes]
27.
28.        return res
29.
30.    def draw(self, frame, det):
31.        vis = frame.copy()
32.        for idx, (fx, fy, fw, fh) in enumerate(det['faces']):
33.            cv2.rectangle(vis, (fx, fy), (fx + fw, fy + fh), (0, 255, 0), 2)
34.            cv2.putText(vis, f"Face #{idx+1}", (fx, fy - 8),
35.                        cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 1)
36.
37.            for ex, ey, ew, eh in det['eyes'].get(idx, []):
38.                cv2.rectangle(vis, (ex, ey), (ex + ew, ey + eh), (255, 100, 0), 2)
39.
40.        return vis
```

Before applying the detection algorithm, the *detect()* method transforms the input BGR image into a single-channel grayscale format and immediately applies `cv2.equalizeHist()`. Histogram equalization uniformly distributes the intensity values,

significantly enhancing the local contrast of the facial topography. This preprocessing step is vital for ensuring that the Haar Cascade classifiers operate effectively, even under poor or uneven ambient illumination.

The method then employs *detectMultiScale()* using a pre-trained frontal face classifier. It returns the exact spatial bounding box (*x, y, width, height*) of the face. To further map the facial landmarks, a secondary eye detection cascade is executed strictly within the upper 60% of the detected facial bounding box (*gray[fy:fy + int(fh * 0.6), ...]*). This targeted search dramatically accelerates computation time and eliminates false positive eye detections in the lower facial regions such as nostrils or mouth features.

3. Region of Interest Extraction (*roi_selector.py*)

The *MultiROIExtractor* class bridges the gap between the spatial domain and the temporal domain. It extracts the color data from specific facial regions and buffers them chronologically.

```

1. class MultiROIExtractor:
2.     PRESETS = {
3.         'forehead': (0.25, 0.05, 0.50, 0.20),
4.         'cheek_left': (0.05, 0.45, 0.35, 0.25),
5.         'cheek_right': (0.60, 0.45, 0.35, 0.25)
6.     }
7.
8.     ROI_COLORS = {
9.         'forehead': (0, 0, 255),
10.        'cheek_left': (255, 100, 0),
11.        'cheek_right': (255, 200, 0)
12.    }
13.
14.    def __init__(self, buffer_seconds=10.0, fps=30.0):
15.        self.fps = fps
16.        self.buffer_len = int(buffer_seconds * fps)
17.        self.buffers = {}
18.
19.        for mode in ('A_forehead', 'B_cheek', 'C_combined'):
20.            self.buffers[mode] = {k: deque(maxlen=self.buffer_len) for k in ('R', 'G', 'B',
'time')}
21.
22.            print(f"[MultiROIExtractor] buffer={buffer_seconds}s x {fps}fps =
{self.buffer_len} samples/mode")
23.
24.    def _coords(self, face_bbox, roi_name):
25.        fx, fy, fw, fh = face_bbox
26.        xf, yf, wf, hf = self.PRESETS[roi_name]
27.        return (
28.            int(fx + xf * fw),
29.            int(fy + yf * fh),
30.            max(int(wf * fw), 10),
31.            max(int(hf * fh), 10)

```

```

32.     )
33.
34.     def _patch_mean(self, frame, face_bbox, roi_name):
35.         h, w = frame.shape[:2]
36.         rx, ry, rw, rh = self._coords(face_bbox, roi_name)
37.         patch = frame[ry:min(ry + rh, h), rx:min(rx + rw, w)]
38.         return patch.mean(axis=(0, 1)) if patch.size > 0 else None
39.
40.     def update(self, frame, face_bbox, timestamp=None):
41.         if timestamp is None:
42.             timestamp = time.time()
43.
44.         fh = self._patch_mean(frame, face_bbox, 'forehead')
45.         cl = self._patch_mean(frame, face_bbox, 'cheek_left')
46.         cr = self._patch_mean(frame, face_bbox, 'cheek_right')
47.
48.         if fh is None and cl is None and cr is None:
49.             return
50.
51.         if fh is not None:
52.             self._push('A_forehead', fh, timestamp)
53.
54.         vck = [x for x in [cl, cr] if x is not None]
55.         if vck:
56.             self._push('B_cheek', np.mean(vck, axis=0), timestamp)
57.
58.         all_r = [x for x in [fh, cl, cr] if x is not None]
59.         if all_r:
60.             self._push('C_combined', np.mean(all_r, axis=0), timestamp)
61.
62.     def _push(self, mode, bgr, ts):
63.         b = self.buffers[mode]
64.         b['B'].append(bgr[0])
65.         b['G'].append(bgr[1])
66.         b['R'].append(bgr[2])
67.         b['time'].append(ts)
68.
69.     def get_signals(self, mode):
70.         b = self.buffers[mode]
71.         return {k: np.array(b[k], dtype=np.float64) for k in ('R', 'G', 'B', 'time')} |
72.         {'n_samples': len(b['R'])}
73.
74.     def is_ready(self, mode, min_sec=5.0):
75.         return len(self.buffers[mode]['R']) >= int(min_sec * self.fps)
76.
77.     def reset(self):

```

```

77.     for m in self.buffers:
78.         for k in self.buffers[m]:
79.             self.buffers[m][k].clear()
80.     print("[MultiROIExtractor] All buffers reset.")
81.
82.     def draw(self, frame, face_bbox):
83.         h, w = frame.shape[:2]
84.         for name, color in self.ROI_COLORS.items():
85.             rx, ry, rw, rh = self._coords(face_bbox, name)
86.             cv2.rectangle(frame, (rx, ry), (min(rx + rw, w), min(ry + rh, h)), color, 2)
87.             lbl = {'forehead': 'FH', 'cheek_left': 'CL', 'cheek_right': 'CR'}[name]
88.             cv2.putText(frame, lbl, (rx + 3, ry + 13), cv2.FONT_HERSHEY_SIMPLEX,
189.                 0.38, color, 1)
89.     return frame

```

The `_coords()` method dynamically calculates the precise (x, y) coordinates of the Forehead, Left Cheek, and Right Cheek. Instead of using static pixel counts, it uses proportional multipliers defined in the *PRESETS* dictionary relative to the detected facial bounding box (fw, fh). This dynamic scaling ensures that the ROI correctly encompasses the desired subcutaneous vascular beds regardless of how close or far the subject is from the camera.

The `_patch_mean()` method executes the crucial spatial averaging operation. By slicing the numpy array to isolate the ROI (*patch*), and invoking `.mean(axis=(0, 1))`, it calculates a single RGB triplet that represents the arithmetic mean of all pixels inside that region. As established in the theoretical framework, summing thousands of pixels effectively neutralizes independent Gaussian shot noise inherent to the camera sensor. These pristine RGB averages are then continuously appended into a fixed-length *collections.deque* buffer, formally constructing the raw time-series matrices required for DSP.

4. Digital Signal Processing (*signal_proc.py*)

This script contains two classes: *SignalPreprocessor* and *SGFilter*, representing the temporal cleansing operations that eliminate motion and illumination noise.

```

1. class SignalPreprocessor:
2.     def __init__(self, fps=30.0, low_hz=0.75, high_hz=2.5):
3.         self.fps = fps
4.         self.low_hz = low_hz
5.         self.high_hz = high_hz
6.
7.         # Kalkulasi koefisien filter Biquad
8.         K = 2.0 * fps
9.         w1 = 2.0 * fps * math.tan(math.pi * low_hz / fps)
10.        wh = 2.0 * fps * math.tan(math.pi * high_hz / fps)
11.        w0 = math.sqrt(w1 * wh)
12.        BW = wh - w1
13.        D = K**2 + BW * K + w0**2
14.
15.        self.b0 = (BW * K) / D

```

```

16.     self.b1 = 0.0
17.     self.b2 = -(BW * K) / D
18.
19.     self.a1 = (2 * w0**2 - 2 * K**2) / D
20.     self.a2 = (K**2 - BW * K + w0**2) / D
21.
22.     print(f"[SignalPreprocessor] IIR Biquad BPF [{low_hz}-{high_hz} Hz]")
23.     print(f" b=[{self.b0:.6f}, 0, {self.b2:.6f}] a=[1, {self.a1:.6f}, {self.a2:.6f}]")
24.
25.     def _fwd(self, x):
26.         N = len(x)
27.         y = np.zeros(N)
28.         x1 = x2 = y1 = y2 = 0.0
29.
30.         for n in range(N):
31.             yn = self.b0 * x[n] + self.b1 * x1 + self.b2 * x2 - self.a1 * y1 - self.a2 * y2
32.             x2 = x1
33.             x1 = x[n]
34.             y2 = y1
35.             y1 = yn
36.             y[n] = yn
37.
38.         return y
39.
40.     def filter(self, sig):
41.         if len(sig) < 6:
42.             return sig
43.         # Zero-phase filtering (forward-backward)
44.         y = self._fwd(sig - sig.mean())
45.         return self._fwd(y[::-1])[::-1]
46.
47.     def normalize(self, sig):
48.         p = np.max(np.abs(sig))
49.         return sig / p if p >= 1e-10 else sig

```

The *SignalPreprocessor* implements an Infinite Impulse Response (IIR) Biquad Bandpass Filter. During initialization, it calculates the feedforward (*b*) and feedback (*a*) coefficients based on the user-defined cardiovascular frequency boundaries (0.75 Hz to 2.5 Hz). The *filter()* method exhibits a highly sophisticated zero-phase filtering technique. It processes the signal forward (*self._fwd(sig)*), reverses the output array (*y[::-1]*), processes it forward again, and reverses it back. This mechanism perfectly negates the nonlinear phase delay inherent in standard IIR filters, ensuring that the temporal locations of the blood volume pulses are not artificially shifted.

```

1. class SGFilter:
2.     def __init__(self, window_len=9, poly_order=3):
3.         if window_len % 2 == 0:
4.             window_len += 1

```

```

5.     if poly_order >= window_len:
6.         raise ValueError("poly_order harus < window_len")
7.
8.     self.window_len = window_len
9.     self.poly_order = poly_order
10.    self.half_w = window_len // 2
11.    self.kernel = self._compute_kernel(window_len, poly_order)
12.
13.        print(f"[SGFilter]   window={window_len},   poly={poly_order},
kernel_sum={self.kernel.sum():.6f}")
14.
15.    def _compute_kernel(self, wl, po):
16.        half = wl // 2
17.        J = np.zeros((wl, po + 1), dtype=np.float64)
18.
19.        for i in range(wl):
20.            xi = float(i - half)
21.            for k in range(po + 1):
22.                J[i, k] = xi**k
23.
24.        JtJ = np.zeros((po + 1, po + 1), dtype=np.float64)
25.        for a in range(po + 1):
26.            for b in range(po + 1):
27.                s = 0.0
28.                for i in range(wl):
29.                    s += J[i, a] * J[i, b]
30.                JtJ[a, b] = s
31.
32.        JtJ_inv = self._gauss_inv(JtJ)
33.
34.        h = np.zeros(wl, dtype=np.float64)
35.        for i in range(wl):
36.            for k in range(po + 1):
37.                h[i] += JtJ_inv[0, k] * J[i, k]
38.
39.        return h
40.
41.    def _gauss_inv(self, A):
42.        n = A.shape[0]
43.        aug = np.hstack([A.copy(), np.eye(n, dtype=np.float64)])
44.
45.        for col in range(n):
46.            mr = col + int(np.argmax(np.abs(aug[col:, col])))
47.            if mr != col:
48.                aug[[col, mr]] = aug[[mr, col]]
49.            p = aug[col, col]

```

```

50.         if abs(p) < 1e-14:
51.             raise ValueError("[SGFilter] JtJ singular")
52.         aug[col] /= p
53.
54.         for row in range(n):
55.             if row != col:
56.                 aug[row] -= aug[row, col] * aug[col]
57.
58.     return aug[:, n:]
59.
60. def _convolve(self, signal):
61.     N = len(signal)
62.     hw = self.half_w
63.     padded = np.empty(N + 2 * hw, dtype=np.float64)
64.
65.     # Edge padding dengan reflection
66.     padded[hw:hw + N] = signal
67.     padded[:hw] = signal[1:hw + 1][::-1]
68.     padded[hw + N:] = signal[N - hw - 1:N - 1][::-1]
69.
70.     out = np.zeros(N, dtype=np.float64)
71.     wl = self.window_len
72.     for n in range(N):
73.         s = 0.0
74.         for k in range(wl):
75.             s += self.kernel[k] * padded[n + k]
76.         out[n] = s
77.
78.     return out
79.
80. def smooth(self, signal):
81.     if len(signal) < self.window_len:
82.         return signal
83.     return self._convolve(signal)

```

The *SGFilter* generates the convolution coefficients for Savitzky-Golay smoothing. The `_compute_kernel()` function dynamically constructs the Vandermonde matrix J representing a local polynomial of degree `po`. It then computes the pseudo-inverse matrix $(J^T J)^{-1} J^T$ using a custom Gaussian elimination method (`_gauss_inv`). The resulting 1D kernel (h) acts as a least-squares polynomial curve fitter. When convolved over the time-series signal, it heavily attenuates high-frequency jitter without flattening the sharp, morphological systolic peaks of the cardiac cycle, preserving crucial data for the upcoming peak detection algorithms.

5. Chrominance Transformation (*rppg.py*)

The *rPPGProcessor* encapsulates the mathematical projection models that translate the purified RGB waveforms into a single blood volume pulse signal.

```

1. class rPPGProcessor:

```



```

2.  def __init__(self, method='chrom'):
3.      if method not in ('chrom', 'green'):
4.          raise ValueError(f"Method '{method}' tidak didukung.")
5.      self.method = method
6.      print(f"[rPPGProcessor] Metode: {method.upper()}")
7.
8.  def compute(self, signals):
9.      R = signals['R'].astype(np.float64)
10.     G = signals['G'].astype(np.float64)
11.     B = signals['B'].astype(np.float64)
12.
13.     if len(R) < 10:
14.         return np.zeros(len(R))
15.
16.     if self.method == 'green':
17.         return G - G.mean()
18.
19.     # CHROM Algorithm (De Haan and Jeanne, 2013)
20.     eps = 1e-8
21.     lum = R + G + B + eps
22.     Rn, Gn, Bn = R/lum, G/lum, B/lum
23.
24.     Xs = 3*Rn - 2*Gn
25.     Ys = 1.5*Rn + Gn - 1.5*Bn
26.
27.     # Alpha Projection
28.     return Xs - (np.std(Xs) / (np.std(Ys) + eps)) * Ys

```

The *compute()* method implements the CHROM (Chrominance-based) algorithm. Initially, it normalizes the R, G, and B arrays by dividing them by the total instantaneous luminance (*lum*). This prevents large-scale brightness changes, such as a cloud passing over the sun from skewing the data. Next, the normalized signals are mapped into an orthogonal chrominance space via the equations X_s and Y_s .

The genius of the CHROM method lies in its final subtraction. The standard deviation ratio $\alpha = \sigma(X_s)/\sigma(Y_s)$ serves as an adaptive scaling factor. Because specular reflection (surface glare) affects all camera color channels uniformly, scaling Y_s by α and subtracting it from X_s perfectly cancels out the ambient specular noise. The returned 1D array is a highly refined rPPG waveform reflecting purely the subcutaneous diffuse absorption changes caused by circulating hemoglobin.

6. Frequency Analysis and Tracking (*hr_estimator.py*)

This script translates the continuous waveform into concrete Heart Rate (BPM) values, utilizing the *HeartRateEstimator* for spectral extraction and *KalmanBPM* for temporal tracking.

```

1.  class HeartRateEstimator:
2.      def __init__(self, fps=30.0, bpm_min=45.0, bpm_max=150.0,
3.          conf_threshold=0.05, stability_window=8,
4.          kalman_q_bpm=0.5, kalman_q_trend=0.1, kalman_r=8.0):

```

```

5.     self.fps = fps
6.     self.bpm_min = bpm_min
7.     self.bpm_max = bpm_max
8.     self.conf_threshold = conf_threshold
9.
10.    self._bpm_hist = deque(maxlen=stability_window)
11.    self._conf_hist = deque(maxlen=stability_window)
12.
13.    dt_upd = 15 / fps
14.    self.kalman = KalmanBPM(
15.        q_bpm=kalman_q_bpm,
16.        q_trend=kalman_q_trend,
17.        r_meas=kalman_r,
18.        dt=dt_upd
19.    )
20.
21.    def _hanning(self, N):
22.        return np.array([0.5 * (1 - math.cos(2 * math.pi * n / (N - 1))) for n in range(N)])
23.
24.    def _dft_naive(self, x):
25.        N = len(x)
26.        X = np.zeros(N, dtype=complex)
27.        c = 2 * math.pi / N
28.        for k in range(N):
29.            for n in range(N):
30.                a = c * k * n
31.                X[k] += x[n] * complex(math.cos(a), -math.sin(a))
32.        return X
33.
34.    def _fft(self, x):
35.        N = len(x)
36.        if N == 1:
37.            return np.array([complex(x[0], 0)])
38.        if N % 2 != 0:
39.            return self._dft_naive(x)
40.
41.        E = self._fft(x[0::2])
42.        O = self._fft(x[1::2])
43.        h = N // 2
44.        tw = np.array([complex(math.cos(-2 * math.pi * k / N), math.sin(-2 * math.pi *
k / N)) for k in range(h)])
45.
46.        T = tw * O
47.        X = np.empty(N, dtype=complex)
48.        X[:h] = E + T
49.        X[h:] = E - T

```

```

50.     return X
51.
52.     def _next_pow2(self, n):
53.         p = 1
54.         while p < n:
55.             p <=<= 1
56.         return p
57.
58.     def _fft_mags(self, sig):
59.         N = len(sig)
60.         Nf = self._next_pow2(N)
61.         p = np.zeros(Nf)
62.         p[:N] = sig * self._hanning(N)
63.         X = self._fft(p)
64.         half = Nf // 2
65.         freqs = np.array([k * self.fps / Nf for k in range(half)])
66.         return freqs, np.abs(X[:half])
67.
68.     def estimate_fft(self, sig):
69.         if len(sig) < 8:
70.             return self._empty()
71.
72.         freqs, mags = self._fft_mags(sig)
73.         v = (freqs >= self.bpm_min / 60) & (freqs <= self.bpm_max / 60)
74.
75.         if not v.any():
76.             return self._empty()
77.
78.         mv, fv = mags[v], freqs[v]
79.         pi = np.argmax(mv)
80.         pf = fv[pi]
81.         bpm = pf * 60
82.
83.         conf = float(mv[pi]) / (float(mv.sum()) + 1e-8)
84.         bpm_med = self._iqr_median(bpm, conf)
85.
86.         bpm_k = None
87.         if bpm_med is not None:
88.             bpm_k = self.kalman.update(bpm_med, confidence=conf)
89.             bpm_k = float(np.clip(bpm_k, self.bpm_min, self.bpm_max))
90.
91.         return {
92.             'bpm': float(bpm),
93.             'bpm_median': float(bpm_med) if bpm_med else None,
94.             'bpm_kalman': bpm_k,
95.             'frequencies': freqs,

```

```

96.         'magnitudes': mags,
97.         'peak_freq': float(pf),
98.         'confidence': conf,
99.         'kalman_trend': self.kalman.trend
100.     }
101.
102.     def estimate_peaks(self, sig):
103.         if len(sig) < 10:
104.             return {'bpm': None, 'peak_locs': [], 'ibi_mean': None, 'n_peaks': 0}
105.
106.         md = max(int(self.fps / (self.bpm_max / 60)), 1)
107.         ar = np.max(sig) - np.min(sig)
108.         pks, _ = find_peaks(sig, distance=md, prominence=0.05 * ar)
109.
110.         if len(pks) < 2:
111.             return {'bpm': None, 'peak_locs': pks.tolist(), 'ibi_mean': None, 'n_peaks':
len(pks)}
112.
113.         ibi = np.mean(np.diff(pks)) / self.fps
114.         bpm = 60 / ibi
115.
116.         if not (self.bpm_min <= bpm <= self.bpm_max):
117.             bpm = None
118.
119.         return {
120.             'bpm': float(bpm) if bpm else None,
121.             'peak_locs': pks.tolist(),
122.             'ibi_mean': float(ibi),
123.             'n_peaks': len(pks)
124.         }
125.
126.     def _iqr_median(self, bpm, conf):
127.         if conf >= self.conf_threshold:
128.             self._bpm_hist.append(bpm)
129.             self._conf_hist.append(max(conf, 1e-6))
130.
131.         if len(self._bpm_hist) == 0:
132.             return None
133.         if len(self._bpm_hist) == 1:
134.             return float(self._bpm_hist[0])
135.
136.         ba, ca = np.array(self._bpm_hist), np.array(self._conf_hist)
137.         q1, q3 = np.percentile(ba, 25), np.percentile(ba, 75)
138.         iqr = q3 - q1
139.         mask = (ba >= q1 - 1.5 * iqr) & (ba <= q3 + 1.5 * iqr)
140.

```

```

141.         bc, cc = (ba[mask], ca[mask]) if mask.any() else (ba, ca)
142.         ws = cc / cc.sum() if cc.sum() > 1e-8 else np.ones(len(cc)) / len(cc)
143.
144.         si = np.argsort(bc)
145.         mid = min(int(np.searchsorted(np.cumsum(ws[si]), 0.5)), len(bc) - 1)
146.         return float(bc[si[mid]])
147.
148.     def _empty(self):
149.         return {
150.             'bpm': None,
151.             'bpm_median': None,
152.             'bpm_kalman': None,
153.             'frequencies': np.array([]),
154.             'magnitudes': np.array([]),
155.             'peak_freq': None,
156.             'confidence': 0.0, 'kalman_trend': 0.0
157.         }
158.
159.     def reset(self):
160.         self._bpm_hist.clear(); self._conf_hist.clear(); self.kalman.reset()

```

The *estimate_fft()* method transforms the rPPG signal into the frequency domain. It calls an internal Fast Fourier Transform function (*_fft_mags*) which automatically applies a Hanning window and zero-pads the signal to the nearest power of two, minimizing spectral leakage. The algorithm then isolates the valid frequency band (*v*) dictated by human physiological limits (e.g., 0.75 Hz to 2.5 Hz). The frequency bin with the absolute maximum magnitude (*np.argmax(mv)*) is identified as the true pulse frequency, which is subsequently multiplied by 60 to yield the raw BPM. A critical *confidence* metric is also computed by taking the ratio of the peak's magnitude to the total spectral energy in the valid band.

```

1.  class KalmanBPM:
2.      def __init__(self, q_bpm=0.5, q_trend=0.1, r_meas=8.0, dt=0.5):
3.          self.dt = dt
4.          self.F = np.array([[1.0, dt], [0.0, 1.0]], dtype=np.float64)
5.          self.H = np.array([[1.0, 0.0]], dtype=np.float64)
6.          self.Q = np.array([[q_bpm, 0.0], [0.0, q_trend]], dtype=np.float64)
7.          self.R = float(r_meas)
8.          self.x = None
9.          self.P = None
10.         self._init = False
11.         print(f"[KalmanBPM] q_bpm={q_bpm}, q_trend={q_trend}, r={r_meas},
            dt={dt}s")
12.
13.     def _init_state(self, bpm):
14.         self.x = np.array([float(bpm), 0.0], dtype=np.float64)
15.         self.P = np.array([self.R * 2, 0.0], [0.0, 1.0], dtype=np.float64)
16.         self._init = True

```

```

17.
18. def update(self, z_bpm, confidence=1.0):
19.     if not self._init:
20.         self._init_state(z_bpm)
21.         return z_bpm
22.
23.     # Predict step
24.     x_pred = self.F @ self.x
25.     P_pred = self.F @ self.P @ self.F.T + self.Q
26.
27.     # Adaptive Measurement Noise
28.     R_a = self.R / max(float(confidence), 0.01)
29.     z = np.array([[float(z_bpm)]])
30.
31.     # Update step
32.     y_inn = z - self.H @ x_pred
33.     S = self.H @ P_pred @ self.H.T + np.array([[R_a]])
34.     K = P_pred @ self.H.T * (1.0 / S[0, 0])
35.
36.     self.x = x_pred + K * y_inn[0, 0]
37.     self.P = (np.eye(2) - K @ self.H) @ P_pred
38.
39.     return float(self.x[0, 0])
40.
41. def reset(self):
42.     self.x = None
43.     self.P = None
44.     self._init = False
45.     print("[KalmanBPM] reset.")
46.
47. @property
48. def bpm_estimate(self):
49.     return float(self.x[0, 0]) if self._init else None
50.
51. @property
52. def trend(self):
53.     return float(self.x[1, 0]) if self._init else 0.0

```

The raw BPM is often subject to erratic, sudden jumps. The *KalmanBPM.update()* method rectifies this by employing a 2-State Constant Velocity Kalman Filter. It predicts the current state vector x that containing the BPM and its trend/velocity, using a linear kinematic model or the F matrix.

The most advanced feature of this implementation is the adaptive measurement noise covariance, R_a . By dividing the base measurement variance parameter R by the FFT's confidence score, the system dynamically alters its trust in the new measurement z . If the FFT indicates a highly noisy signal (low confidence), R_a skyrockets. Consequently, the Kalman Gain (K) approaches zero, and the system relies entirely on its previous trend

predictions to maintain stability. Conversely, a high confidence score forces the system to quickly adapt to the new, accurate BPM reading, resulting in a medical-grade, fluid tracking behavior.

2.2.2. Monitor Modules

The ``monitor/`` directory manages the output and visualization layers of the pipeline. In real-time computer vision applications, rendering complex graphs concurrently with video processing often creates severe bottlenecks, leading to frame drops or GUI freezing. The modules within this directory are explicitly engineered to decouple the analytical visualization and data serialization processes from the main application loop, ensuring continuous, high-speed execution.

1. Real-Time Visualization (*plotter.py*)

The *MultiROIPlotter* class is responsible for generating the live, 12-panel dashboard that displays the time-domain signals, filtered channels, and real-time heart rate metrics. Instead of using Matplotlib's interactive backends (like TkAgg or Qt5Agg) which notoriously block the main execution thread during ``flush_events()``, this class leverages the non-interactive ``FigureCanvasAgg`` backend.

```
1. class MultiROIPlotter:
2.     MODE_LABELS = {
3.         'A_forehead': 'Mode A - Forehead',
4.         'B_cheek': 'Mode B - Cheek',
5.         'C_combined': 'Mode C - Combined'
6.     }
7.
8.     MODE_HDR_C = {
9.         'A_forehead': '#ff6655',
10.        'B_cheek': '#55aaff',
11.        'C_combined': '#88ee55'
12.    }
13.
14.    CH_COLORS = {
15.        'R': ((1.0, 0.55, 0.55), (1.0, 0.1, 0.1)),
16.        'G': ((0.4, 0.9, 0.5), (0.1, 0.85, 0.2)),
17.        'B': ((0.45, 0.65, 1.0), (0.15, 0.4, 1.0))
18.    }
19.
20.    RPPG_C = {
21.        'A_forehead': (1.0, 0.4, 0.4),
22.        'B_cheek': (0.3, 0.65, 1.0),
23.        'C_combined': (0.4, 0.95, 0.4)
24.    }
25.
26.    PEAK_C = (1.0, 0.95, 0.2)
27.    FS = {'title': 11, 'axis': 10, 'tick': 9, 'legend': 8}
28.
29.    def __init__(self, window_name='rPPG Multi-ROI Comparison (A|B|C)':
30.        self.window_name = window_name
```

```

31.     self._init_agg(window_name)
32.
33.     def _init_agg(self, window_name):
34.         self._agg_w = 2400
35.         self._agg_h = 1500
36.         self._disp_w = 1440
37.         self._disp_h = 900
38.
39.         self.fig = Figure(figsize=(24, 15), dpi=100, facecolor='#12121e')
40.         self.canvas = FigureCanvasAgg(self.fig)
41.         self.axes = {}
42.         modes = ['A_forehead', 'B_cheek', 'C_combined']
43.
44.         for col, mode in enumerate(modes):
45.             for row in range(4):
46.                 ax = self.fig.add_subplot(4, 3, row * 3 + col + 1)
47.                 ax.set_facecolor('#0c0c18')
48.                 ax.tick_params(colors='#aaaaaa', labelsize=self.FS['tick'])
49.                 for sp in ax.spines.values():
50.                     sp.set_edgecolor('#3a3a4a')
51.                 self.axes[(row, col)] = ax
52.
53.         self.fig.tight_layout(pad=2.0, h_pad=1.5, w_pad=1.2)
54.
55.         self._render_lock = threading.Lock()
56.         self._plot_img = None
57.         self._render_busy = False
58.
59.         cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)
60.         cv2.resizeWindow(window_name, self._disp_w, self._disp_h)
61.         print(f"[MultiROIPlotter] Mode: Agg HQ | {self._agg_w}x{self._agg_h}px ->
{self._disp_w}x{self._disp_h}px")
62.
63.         def update(self, data, fps):
64.             if self._render_busy:
65.                 return # skip if frame is still rendering
66.             self._render_busy = True
67.             threading.Thread(target=self._render_worker_agg, args=(data, fps),
daemon=True).start()
68.
69.         def show_if_ready(self):
70.             """Display the latest rendered image if available."""
71.             with self._render_lock:
72.                 if self._plot_img is not None:
73.                     cv2.imshow(self.window_name, self._plot_img)
74.

```



```

75. def _render_worker_agg(self, data, fps):
76.     try:
77.         self._render_subplots(data, fps)
78.         self.canvas.draw()
79.         buf = self.canvas.buffer_rgba()
80.         img = np.asarray(buf, dtype=np.uint8).reshape(self._agg_h, self._agg_w, 4)
81.         bgr = cv2.cvtColor(img, cv2.COLOR_RGBA2BGR)
82.         bgr_disp = cv2.resize(bgr, (self._disp_w, self._disp_h),
            interpolation=cv2.INTER_AREA)
83.
84.         with self._render_lock:
85.             self._plot_img = bgr_disp
86.         # ... (error handling and state reset)

```

The *update()* method implements a non-blocking concurrency model. When invoked, it immediately spawns a lightweight background thread daemon to handle the mathematically heavy *canvas.draw()* operation. A state flag (*self._render_busy*) is utilized as a rapid concurrency guard; if the plotter is still processing the previous frame's data, the new update request is simply discarded. This ensures that the system's memory and CPU are not overwhelmed by a backlog of render requests, maintaining the integrity of the camera's FPS.

Inside the *_render_worker_agg()* method, the generated Matplotlib figure is serialized into a raw RGBA pixel buffer. This buffer is then reshaped into a standard numpy array, converted into the BGR color space using *cv2.cvtColor*, and anti-aliased (resized) for display. Finally, a threading lock (*self._render_lock*) is acquired to safely push the rendered frame into a shared variable (*self._plot_img*), which the main GUI loop can fetch and display without interruption.

```

1. def _plot_rppg(self, ax, t, sig, peaks, bpm_k, bpm_r, trend, mode):
2.     ax.clear()
3.     ax.set_facecolor('#0c0c18')
4.     c = self.RPPG_C[mode]
5.     N = len(sig)
6.
7.     ax.plot(t, sig, color=c, linewidth=1.2, label=r'PPG (CHROM+SG)')
8.     ax.axhline(0, color='#333344', linewidth=0.6, linestyle='--')
9.
10.    if peaks:
11.        pk = np.array(peaks)
12.        pk = pk[pk < N]
13.        ax.scatter(t[pk], sig[pk], color=self.PEAK_C, s=30, zorder=5, label='peaks')
14.
15.    hdr = self.MODE_LABELS[mode]
16.    hc = self.MODE_HDR_C[mode]
17.
18.    if bpm_k:
19.        ts = '^' if trend > 0.3 else 'v' if trend < -0.3 else '->'
20.        bstr = f'| Kalman: {bpm_k:.1f} BPM {ts}'

```

```

21.         if bpm_r:
22.             bstr += f' (raw:{bpm_r:.1f})'
23.         else:
24.             bstr = '| buffering...'
25.
26.             ax.set_title(f'{hdr}{bstr}', color=hc, fontsize=self.FS['title'], pad=4,
27. fontweight='bold')
28.             ax.set_ylabel('rPPG (a.u.)', color='#bbbbbb', fontsize=self.FS['axis'],
29. labelpad=4)
30.             ax.legend(fontsize=self.FS['legend'], loc='upper left', facecolor='#1a1a2e',
31. labelcolor='white', framealpha=0.7)
32.             self._style(ax, t)

```

The `_plot_rppg()` method acts as the specific mapping function for the primary blood volume pulse (BVP) waveform. It receives the temporal array (t) and the highly filtered CHROM signal (sig). Utilizing Matplotlib's object-oriented API (ax), it plots the continuous continuous wave and overlays the validated systolic peaks ($peaks$) using discrete scatter points. Furthermore, it dynamically parses the Kalman Filter's states, appending the smoothed BPM (bpm_k) and translating the temporal trend derivative ($trend$) into intuitive directional arrows, providing immediate physiological context directly on the waveform.

2. Data Logging and Serialization (*data_saver.py*)

The *MultiROIDataSaver* class functions as the exhaustive data archiving engine. When the user initiates a save command, this module captures a snapshot of the current temporal memory and exports both the raw numerical datasets and a suite of high-resolution analytical infographics for post-processing and scientific validation.

```

1. class MultiROIDataSaver:
2.     MODE_LABELS = {
3.         'A_forehead': 'Mode A - Forehead',
4.         'B_cheek': 'Mode B - Cheek',
5.         'C_combined': 'Mode C - Combined'
6.     }
7.
8.     MODE_COLORS = {
9.         'A_forehead': '#ff6644',
10.        'B_cheek': '#44aaff',
11.        'C_combined': '#55ee55'
12.    }
13.
14.    CH_STYLE = {
15.        'R': ('#ff9988', '#ff2222'),
16.        'G': ('#88dd99', '#00cc33'),
17.        'B': ('#88aaff', '#2244ff')
18.    }
19.
20.    def __init__(self, output_dir='rppg_output'):
21.        self.output_dir = output_dir

```

```

22.
23. def save_all(self, results, fps, bpm_log=None):
24.     os.makedirs(self.output_dir, exist_ok=True)
25.     ts = time.strftime("%Y%m%d_%H%M%S")
26.
27.     self._save_csv(results, fps, ts)
28.     self._plot_rppg_all(results, fps, ts)
29.     self._plot_fft_overlay(results, ts)
30.     self._plot_bpf_rgb(results, fps, ts)
31.     self._plot_hr_comparison(results, ts)
32.     self._plot_sg_effect(results, fps, ts)
33.     self._plot_channel_fft(results, fps, ts)
34.
35.     if bpm_log:
36.         self._plot_kalman_history(bpm_log, ts)
37.
38.     print(f"[MultiROIDataSaver] All files saved to: '{self.output_dir}'")
39.     return self.output_dir
40.
41. def _save_csv(self, results, fps, ts):
42.     path = os.path.join(self.output_dir, f'rppg_data_{ts}.csv')
43.     Ns = {}
44.     for m, d in results.items():
45.         rs = d.get('raw_signals', {})
46.         Ns[m] = min(
47.             len(rs.get('R', [])), len(rs.get('G', [])), len(rs.get('B', [])),
48.             len(d.get('rppg_norm', [])), len(d.get('filt_r', [])),
49.             len(d.get('filt_g', [])), len(d.get('filt_b', []))
50.         )
51.
52.     N_max = max(Ns.values()) if Ns else 0
53.     if N_max == 0:
54.         print(" CSV -> skip")
55.         return
56.
57.     with open(path, 'w') as f:
58.         cols = ['frame_idx', 'timestamp']
59.         for m in ('A_forehead', 'B_cheek', 'C_combined'):
60.             for ch in ['R_raw', 'G_raw', 'B_raw', 'R_BPF', 'G_BPF', 'B_BPF',
61. 'R_BPFSG', 'G_BPFSG', 'B_BPFSG', 'rPPG']:
62.                 cols.append(f'{m}_{ch}')
63.             f.write(','.join(cols) + '\n')
64.
65.         for i in range(N_max):
66.             t_arr = results.get('A_forehead', {}).get('raw_signals', {}).get('time',
np.array([]))

```

```

66.         row =[str(i), f'{t_arr[i]:.6f}' if i < len(t_arr) else '0']
67.
68.         for m in ('A_forehead', 'B_cheek', 'C_combined'):
69.             d = results.get(m, {})
70.             rs = d.get('raw_signals', {})
71.             v = lambda a: f'{a[i]:.6f}' if i < len(a) else "
72.
73.             row += [
74.                 v(rs.get('R', [])), v(rs.get('G', [])), v(rs.get('B', [])),
75.                 v(d.get('bpf_r', [])), v(d.get('bpf_g', [])), v(d.get('bpf_b', [])),
76.                 v(d.get('filt_r', [])), v(d.get('filt_g', [])), v(d.get('filt_b', [])),
77.                 v(d.get('rppg_norm', []))
78.             ]
79.             f.write(','.join(row) + '\n')
80.         print(f" CSV      -> {path}")
81. # ...

```

The `_save_csv()` function is arguably the most critical component for data reproducibility. It systematically traverses the buffered memory of all three operational modes (Forehead, Cheek, Combined). For every discrete temporal frame (i), it extracts the exact timestamp and pulls ten specific data points per mode: the raw RGB values, the BPF-filtered RGB values, the Savitzky-Golay smoothed RGB values, and the final normalized CHROM rPPG signal. By writing these arrays row-by-row into a Comma-Separated Values (CSV) format, the system ensures that the entire mathematical evolution of the signal, from raw photometric noise to the purified physiological wave, is permanently preserved for future verification.

```

1. def _plot_kalman_history(self, bpm_log, ts):
2.     # ... (setup figure and loop through modes)
3.     ts_arr = np.array([e['timestamp'] for e in log])
4.     br_arr = np.array([e['bpm_raw'] or 0 for e in log])
5.     bm_arr = np.array([e['bpm_median'] or 0 for e in log])
6.     bk_arr = np.array([e['bpm_kalman'] or 0 for e in log])
7.     cf_arr = np.array([e['confidence'] for e in log])
8.     t_rel = ts_arr - ts_arr[0]
9.
10.    ax.fill_between(t_rel, 45, 45 + cf_arr * 100, alpha=0.08, color=c,
11.                    label='Confidence x100')
12.    ax.scatter(t_rel[valid_r], br_arr[valid_r], color='#888888', s=20, alpha=0.5,
13.               label='BPM Raw FFT')
14.    ax.step(t_rel[valid_m], bm_arr[valid_m], color='#ffcc33', where='mid',
15.            label='BPM Median (IQR)')
16.    ax.plot(t_rel[valid_k], bk_arr[valid_k], color=c, linewidth=2.0, label='BPM
17.            Kalman (output)')
18.    # ... (save figure to disk)

```

Beyond standard waveforms, the class generates complex tracking diagnostics, as demonstrated in `_plot_kalman_history()`. This method reconstructs the chronological performance of the Heart Rate Estimator throughout the session. It extracts the raw FFT

output (*br_arr*), the Interquartile Range (IQR) median pre-filter (*bm_arr*), and the final Kalman-smoothed output (*bk_arr*).

By plotting the highly erratic raw FFT points against the smooth Kalman trajectory, the graph visually justifies the necessity of the 2-State tracking algorithm. Furthermore, it overlays the FFT's spectral confidence score (*cf_arr*) as a shaded background region. This visualization clearly demonstrates the correlation between signal clarity (high confidence) and the mathematical convergence of the Kalman Filter, proving the systemic reliability of the entire rPPG pipeline.

2.2.3. Configuration Module (*config.py*)

In complex biomedical imaging applications, hardcoding hyperparameter values directly into the execution scripts is considered a poor software engineering practice. To ensure optimal modularity and scalability, the *config.py* script functions as the centralized hyperparameter registry for the entire rPPG pipeline. It defines a single dictionary, *DEFAULT_CONFIG*, which dictates the initial state of the hardware interfaces, digital signal processing boundaries, and statistical tracking constraints.

```
1. DEFAULT_CONFIG = {  
2.     "camera_index": 0,  
3.     "fps": 30.0,  
4.     "buffer_seconds": 10.0,  
5.     "min_ready_sec": 5.0,  
6.     "update_interval": 15,  
7.     "plot_interval": 15,  
8.     "bpf_low_hz": 0.75,  
9.     "bpf_high_hz": 2.5,  
10.    "sg_window": 9,  
11.    "sg_poly": 3,  
12.    "rppg_method": "chrom",  
13.    "bpm_min": 45.0,  
14.    "bpm_max": 150.0,  
15.    "kalman_q_bpm": 0.5,  
16.    "kalman_q_trend": 0.1,  
17.    "kalman_r": 8.0,  
18.    "output_dir": "rppg_output",  
19.    "auto_save": True,  
20. }
```

The first logical grouping within this configuration dictates the temporal boundaries of the system. Parameters such as *fps* (30.0) establish the expected sampling frequency. The *buffer_seconds* parameter that defaulted to 10.0 seconds is fundamentally critical; it defines the length of the sliding window used for the time-series array. Mathematically, the length of this buffer determines the frequency resolution (Δf) of the subsequent FFT, where $\Delta f = \frac{1}{T}$ and T is the buffer duration. A 10-second buffer provides a sufficiently narrow frequency bin resolution to detect minute variations in the heart rate while ensuring that the system responds dynamically to recent physiological changes.

The second grouping defines the absolute constraints for the mathematical filters. *bpf_low_hz* and *bpf_high_hz* establish the passband cutoffs (0.75 Hz to 2.5 Hz), directly translating to the physiological limit of 45 to 150 Beats Per Minute. The *sg_window* and

sg_poly control the aggressiveness of the Savitzky-Golay smoothing. Furthermore, the Kalman Filter's state covariances are initialized here. *kalman_q_bpm* and *kalman_q_trend* represent the process noise covariance matrix (Q), dictating how much the system trusts its internal kinematic predictions. The *kalman_r* parameter initializes the baseline measurement noise (R). By isolating these variables into *config.py*, researchers can rapidly tune the mathematical sensitivity of the pipeline without altering the core algorithmic logic.

2.2.4. CHROM Method Justification (*chrom_justification.py*)

To scientifically validate the efficacy of the chosen rPPG algorithm, the *chrom_justification.py* script was engineered as a standalone analytical tool. Its primary objective is to benchmark the performance of the Chrominance-based (CHROM) method against naive single-channel extractions (Red, Green, and Blue) and advanced Independent Component Analysis (ICA). This script directly addresses the hypothesis that CHROM is vastly superior at mitigating specular reflection and motion artifacts.

```

1. def _chrom_decompose(R, G, B):
2.     eps = 1e-8
3.     lum = R + G + B + eps
4.     Rn, Gn, Bn = R / lum, G / lum, B / lum
5.     Xs = 3 * Rn - 2 * Gn
6.     Ys = 1.5 * Rn + Gn - 1.5 * Bn
7.     alpha = np.std(Xs) / (np.std(Ys) + eps)
8.     S = Xs - alpha * Ys
9.     def _n(x): p = np.max(np.abs(x)); return x / p if p > 1e-10 else x
10.    return _n(Xs), _n(Ys), _n(S), float(alpha)

```

The *_chrom_decompose()* function is the mathematical epicenter of this validation script. It replicates the CHROM projection algorithm but explicitly returns the intermediate variables, that are the two orthogonal chrominance vectors (X_s and Y_s), the adaptive scaling ratio (α), and the final un-filtered BVP signal (S). By exposing these intermediate variables, the script can visually plot the exact moment the specular noise is canceled out. The calculation of α , the standard deviation ratio between X_s and Y_s , is the critical mechanism that suppresses the white-light ambient interference, allowing the system to verify that S contains purely the diffuse reflection modulated by the subcutaneous blood volume.

```

1. def _fft_analysis(sig, fps, bpm_min=45.0, bpm_max=150.0):
2.     N = len(sig)
3.     win = np.hanning(N)
4.     sig_w = (sig - sig.mean()) * win
5.     Nf = 1
6.     while Nf < N: Nf <= 1
7.     Nf *= 2
8.     X = np.fft.rfft(sig_w, n=Nf)
9.     mags = np.abs(X)
10.    freqs = np.fft.rfftfreq(Nf, d=1.0/fps)
11.    f_lo, f_hi = bpm_min / 60.0, bpm_max / 60.0
12.    band = (freqs >= f_lo) & (freqs <= f_hi)
13.    noise = ~band & (freqs > 0.1) & (freqs < fps/2)

```

```

14. sig_pwr = float(np.sum(mags[band]**2)) + 1e-12
15. noise_pwr = float(np.sum(mags[noise]**2)) + 1e-12
16. snr_db = 10 * np.log10((sig_pwr / max(band.sum(), 1)) / (noise_pwr /
    max(noise.sum(), 1)))
17. pi = np.argmax(mags[band])
18. return freqs, mags, float(snr_db), float(freqs[band][pi]) * 60.0,
    float(freqs[band][pi])

```

To quantitatively compare the five extraction methods, the `_fft_analysis()` function computes the Signal-to-Noise Ratio (SNR) in the frequency domain. It isolates the spectral energy within the valid physiological pulse band (*sig_pwr*) and contrasts it against the spectral energy outside of this band (*noise_pwr*). The ratio is then converted into a standardized logarithmic decibel scale (*snr_db*). A higher SNR mathematically proves that the extracted waveform is dominated by the periodic cardiac signal rather than erratic motion or illumination artifacts.

```

1. def _ica_best_component(R, G, B, fps, bpm_min, bpm_max):
2.     X = np.column_stack([R - R.mean(), G - G.mean(), B - B.mean()])
3.     if _HAS_SKLEARN:
4.         try:
5.             S = FastICA(n_components=3, random_state=0, max_iter=500,
                tol=0.01).fit_transform(X)
6.         except Exception:
7.             _, _, Vt = np.linalg.svd(X, full_matrices=False)
8.             S = X @ Vt.T
9.     else:
10.        _, _, Vt = np.linalg.svd(X, full_matrices=False)
11.        S = X @ Vt.T
12.
13.    best_comp, best_snr = S[:, 0], -999.0
14.    for i in range(S.shape[1]):
15.        comp = S[:, i]
16.        p = np.max(np.abs(comp))
17.        comp_n = comp / p if p > 1e-10 else comp
18.        _, _, snr, _, _ = _fft_analysis(comp_n, fps, bpm_min, bpm_max)
19.        if snr > best_snr:
20.            best_snr, best_comp = snr, comp_n
21.    return best_comp

```

Furthermore, the script exhibits high resilience in its software engineering. For the ICA benchmark (`_ica_best_component`), it attempts to utilize the *FastICA* algorithm from the *scikit-learn* library. However, if the dependency is missing, it seamlessly falls back to an algebraic Principal Component Analysis (PCA) equivalent using Singular Value Decomposition (SVD) via `np.linalg.svd`. Ultimately, the script amalgamates all these analyses into a high-resolution, 5-panel Matplotlib figure that is rendered in the background thread and saved to the disk. This provides an irrefutable, data-driven justification for prioritizing the CHROM method in the main live-monitoring pipeline.

2.2.5. GUI (*main.py*)

The *main.py* script acts as the orchestrator of the entire rPPG system. Built upon the *PyQt6* framework, it seamlessly integrates the rigorous mathematical backend (`'core/'` and `'monitor/'` modules) with a modern, reactive graphical frontend. In heavy computer vision and digital signal processing applications, the primary software engineering challenge is maintaining a fluid UI without interrupting the high-frequency camera capture loop. To achieve this, `'main.py'` extensively utilizes multithreading architectures and event-driven observer patterns.

```

1. class MonitorWorker(QThread):
2.     frame_ready = pyqtSignal(np.ndarray)
3.     bpm_updated = pyqtSignal(dict)
4.     status_changed = pyqtSignal(str)
5.
6.     def __init__(self, cfg: dict, parent=None):
7.         super().__init__(parent)
8.         self._running = False
9.         self._hr_lock = threading.Lock()
10.        # ... (initialization of buffers and states)
11.
12.    def run(self):
13.        # ... (instantiation of Webcam, Extractor, Preprocessor, etc.)
14.        while self._running:
15.            ret, frame = webcam.read_frame()
16.            # ... (Face detection and signal extraction pipeline)
17.            self.frame_ready.emit(vis)
18.            self.bpm_updated.emit(bpm_snapshot)

```

The *MonitorWorker* class inherits from PyQt6's *QThread*, isolating the intensive *while* loop containing the OpenCV frame acquisition, Haar Cascade detection, and the FFT/Kalman calculations into an independent background thread. If these operations were executed on the main application thread, the GUI would completely freeze (block) during computation.

To safely transmit the processed data back to the main thread for rendering, the script employs the Observer pattern via *pyqtSignal*. Instead of returning variables directly, which would violate thread-safety rules, the *MonitorWorker* dynamically emits signals such as *frame_ready* containing the annotated numpy image array, and *bpm_updated* containing the dictionary of estimated heart rates. The main GUI event loop listens for these signals and safely updates the visual elements without interrupting the underlying physiological data extraction process.

```

1. def get_raw_signals_snapshot(self, mode='C_combined'):
2.     with self._hr_lock:
3.         r = self._results.get(mode)
4.         if r is not None and 'raw_signals' in r:
5.             return {
6.                 'R': r['raw_signals']['R'].copy(),
7.                 'G': r['raw_signals']['G'].copy(),
8.                 'B': r['raw_signals']['B'].copy(),
9.             }

```


10. return None

Because the system runs multiple threads concurrently, e.g., the *MonitorWorker* writing the rPPG results and the *JustifyWorker* attempting to read those results for comparative analysis, it is highly susceptible to Race Conditions. A race condition occurs when one thread modifies a memory block while another thread is reading it, leading to corrupted data or fatal memory access violations.

To prevent this, a Mutual Exclusion object (Mutex), implemented via *threading.Lock()*, is utilized. The *get_raw_signals_snapshot()* method uses the *self._hr_lock*: context manager to acquire the lock before accessing the *self._results* dictionary. This mathematically guarantees that no other thread can alter the arrays while the snapshot is being constructed. Furthermore, the method explicitly *calls .copy()* on the numpy arrays to pass by value rather than by reference, strictly isolating the data memory used by the *JustifyWorker* from the live buffering system.

<pre>1. class BpmCard(QFrame): 2. def update_bpm(self, data: dict): 3. # ... (extracting dictionary values) 4. bk, bm, br, bp = data.get('bpm_kalman'), data.get('bpm_median'), data.get('bpm_raw'), data.get('peak_bpm') 5. conf, trend = data.get('confidence', 0.0), data.get('kalman_trend', 0.0) 6. 7. if bk is not None: 8. color = PALETTE['bpm_low'] if bk < 60 else PALETTE['bpm_normal'] if bk <= 100 else PALETTE['bpm_high'] 9. self.lbl_bpm.setText(f'{bk:.1f}') 10. self.lbl_bpm.setStyleSheet(f'color: {color}; font-size: 46px; font-weight: 800; letter-spacing: -1.5px;') 11. 12. # ... (updating detailed string formats) 13. arrow = ("▲ Up" if trend > 0.3 else "▼ Down" if trend < -0.3 else "→ Stable") 14. t_color = "#10B981" if abs(trend) < 0.3 else "#EF4444" 15. self.lbl_trend.setStyleSheet(f'font-size: 11px; font-weight: bold; color: {t_color};') 16. self.lbl_trend.setText(f'Trend: {arrow} ({trend:+.2f}/s)')</pre>

The *BpmCard* class demonstrates a highly reactive UI paradigm. When the main *MainWindow* receives the *bpm_updated* signal from the background thread, it triggers the *update_bpm()* method for each designated Region of Interest (Forehead, Cheek, Combined). The method unpacks the complex dictionary containing the raw FFT BPM (*br*), the median pre-filter (*bm*), the finalized Kalman tracker (*bk*), and the confidence metrics.

The method incorporates conditional styling based on physiological boundaries. If the Kalman-smoothed BPM (*bk*) falls below 60 (Bradycardia) or exceeds 100 (Tachycardia), the CSS stylesheet of the label is dynamically recompiled to alert the user visually with specific palette colors. Similarly, the first-order derivative of the Kalman state, the *trend* variable, indicating the instantaneous acceleration or deceleration of the

heart rate, is parsed into visual arrows and color-coded. This transforms raw algorithmic outputs into an intuitive, medical-grade monitoring dashboard.

2.3. Result of the Program and Analysis

This sub-chapter presents the empirical results generated by the developed remote Photoplethysmography (rPPG) system. The analysis systematically evaluates the graphical user interface, real-time live monitoring capabilities, intermediate digital signal processing outputs, and the final heart rate estimations compared against a clinical ground-truth device.

*Initial GUI

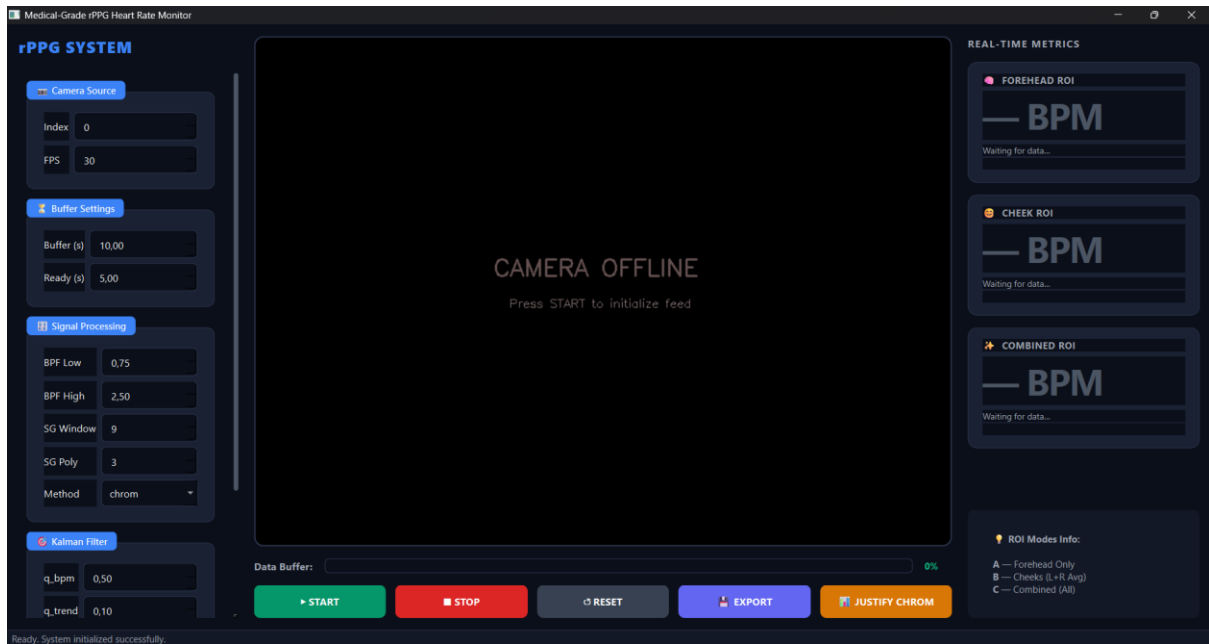


Figure 2.1. Initial GUI Window for the rPPG Camera Program (1)

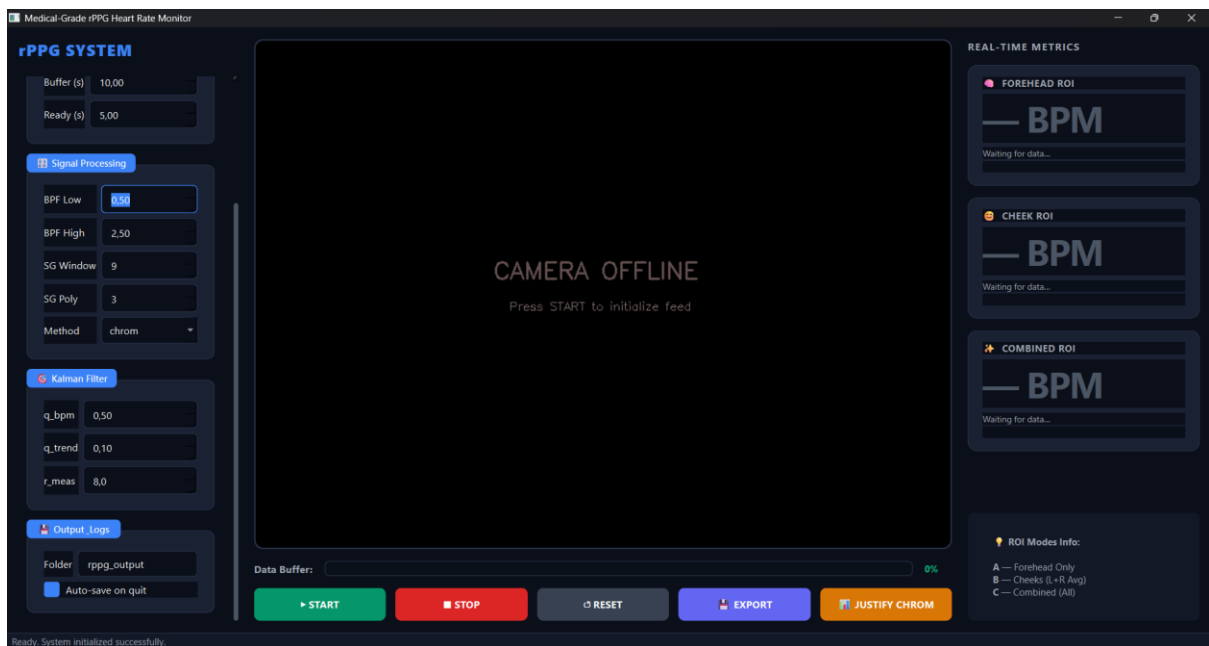


Figure 2.2. Initial GUI Window for the rPPG Camera Program (2)

The first interaction with the system begins with the Graphical User Interface (GUI), built utilizing the PyQt6 framework. As depicted in Figure 2.1 (Initial GUI Window for the rPPG Camera Program 1) and Figure 2.2 (Initial GUI Window for the rPPG Camera Program 2) above, the interface is designed with a modern, dark-themed medical dashboard aesthetic that adhering to the Separation of Concerns (SoC) principle. The interface is structurally divided into three primary vertical sections. The left panel houses the hyperparameter configuration inputs, such as camera index, FPS, buffer lengths, bandpass filter cutoffs, and Kalman filter covariances, which directly map to the variables defined in `config.py`.

The central panel reserves a dedicated frame for the webcam feed, initially displaying a "CAMERA OFFLINE" placeholder. Below this feed lies the control execution row (Start, Stop, Reset, Export, and Justify CHROM) alongside a progress bar indicating the data buffer's fill status. The right panel displays the "Real-Time Metrics" cards, which are currently in a standby state ("Waiting for data..."). This modular layout ensures that the user can configure the rigid mathematical boundaries of the DSP algorithms before the multithreaded *MonitorWorker* initializes the hardware camera, preventing any runtime configuration conflicts.

2.3.1. rPPG Program Live Monitoring

Upon initiating the program, the system begins acquiring frames and executes spatial localization. As shown in Figure 2.3 (Live Monitoring Program Run-time 1), the Viola-Jones Haar Cascade algorithm successfully detects the user's face, indicated by the green bounding box. Subsequently, the *MultiROIExtractor* dynamically maps the Regions of Interest (ROIs). The blue and cyan rectangular overlays represent the targeted extraction zones: the Forehead (Mode A) and the Left/Right Cheeks (Mode B). By restricting the spatial averaging to these specific, highly vascularized areas, the system effectively maximizes the collection of diffuse reflection while minimizing noise from non-skin features like hair or background elements.

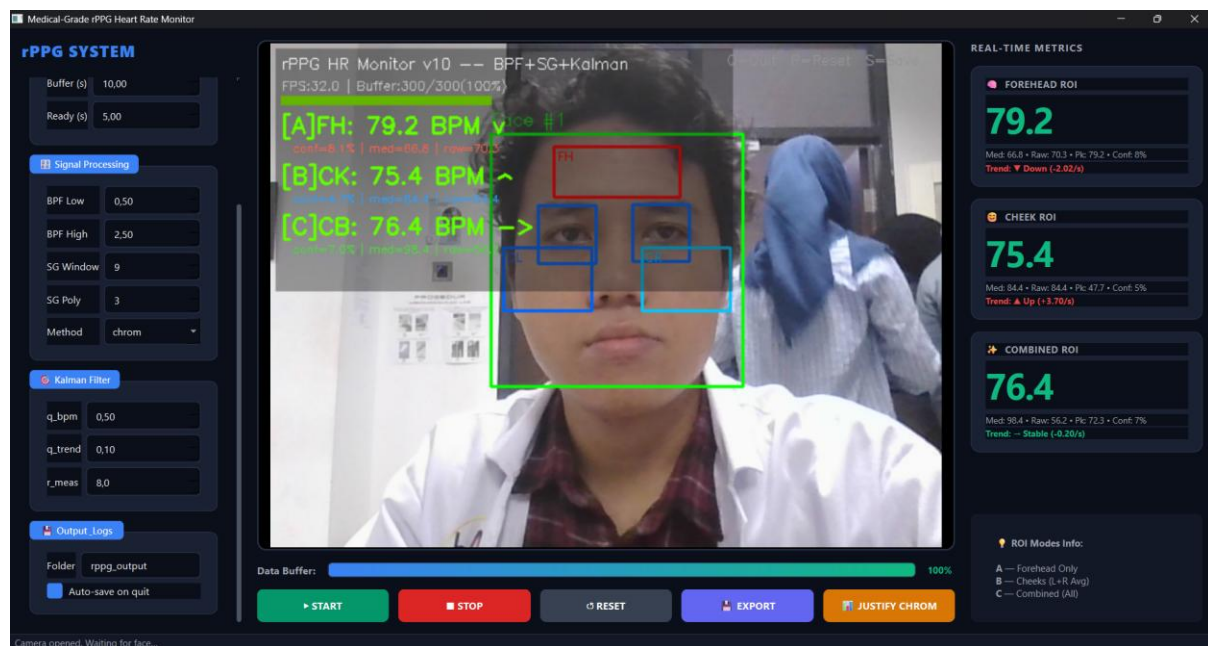


Figure 2.3. Live Monitoring Program Run-time

Simultaneously, the background rendering engine (*MultiROIPlotter*) spawns an external visualization dashboard, presented in Figure 2.4 (Live Monitoring Program Run-time and it Acquired rPPG Signal). This 12-panel Matplotlib figure renders the real-time temporal arrays without blocking the main OpenCV camera loop. The plots actively display the continuous time-series RGB signals fluctuating over the 10-second sliding buffer window. This visual feedback confirms that the spatial averaging technique is successfully transforming the 2D facial pixel matrices into 1D temporal waveforms, serving as the foundational raw data for the ensuing digital signal processing pipeline.



Figure 2.4. Live Monitoring Program Run-time and it Acquired rPPG Signal

2.3.2. rPPG Program Results

The core analytical strength of the system is demonstrated through its automated data logging and visualization exports, which capture the complete mathematical evolution of the cardiovascular signal.



Figure 2.5. Plot BPF IIR combined with Savitzky-Golay Smoothing on RGB Channels in different Mode: A.Forehead, B. Cheek, C. Combined

Temporal Filtering and Smoothing

The impact of the preprocessing stage is vividly illustrated in Figure 2.5 (Plot BPF IIR combined with Savitzky-Golay Smoothing on RGB Channels in different Mode) and Figure 2.11 (Savitzky-Golay Smoothing Effect on different ROI Modes). The raw spatial signals contain severe low-frequency baseline wandering due to respiration and posture shifts, and high-frequency jitter (sensor shot noise). By applying the zero-phase IIR Biquad Bandpass Filter (0.75 Hz - 2.5 Hz), the out-of-band noise is aggressively attenuated. Furthermore, as specifically highlighted in Figure 2.11 for the Green channel, the implementation of the Savitzky-Golay (SG) filter, using a local polynomial regression, expertly smooths the waveform. Unlike a standard moving average that would flatten the data, the SG filter preserves the morphological amplitude and sharpness of the systolic peaks, generating a highly refined periodic wave.

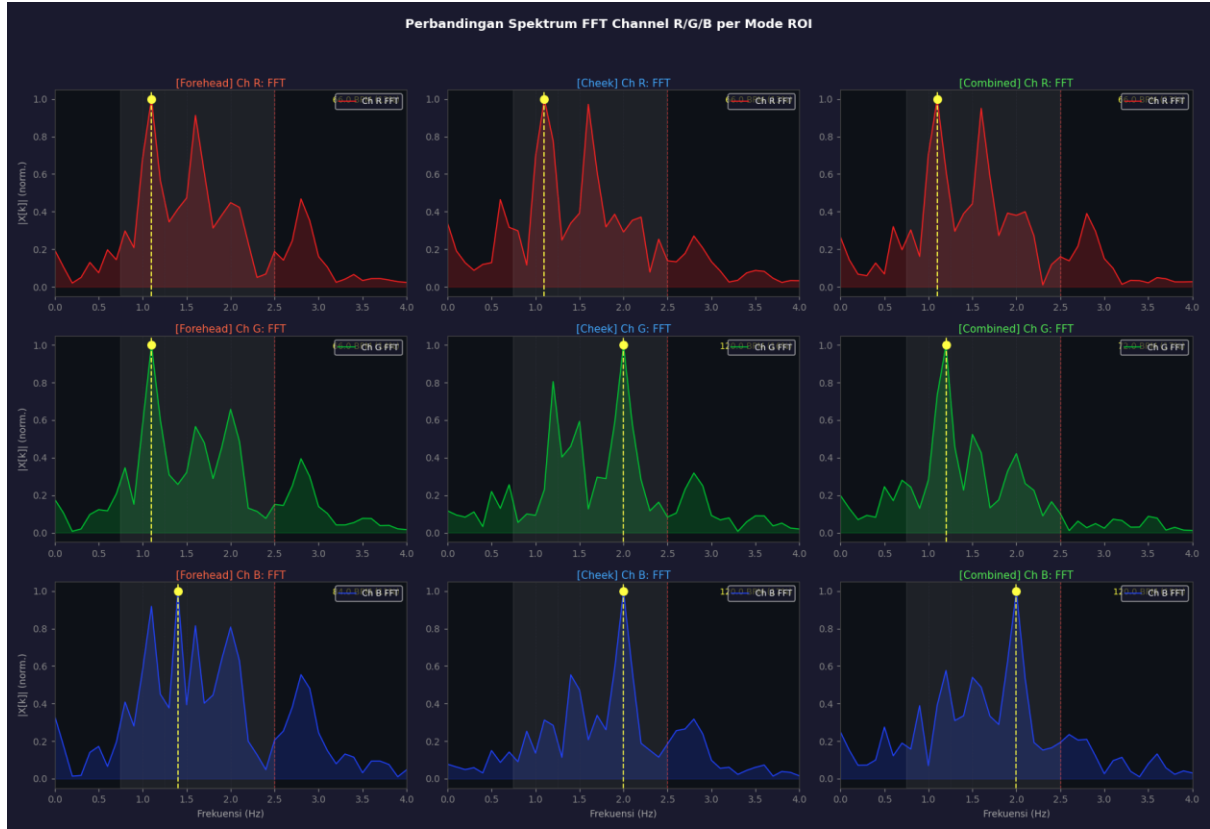


Figure 2.6. Plot FFT Spectrum on RGB Channels in different Mode: A.Forehead, B. Cheek, C. Combined

Frequency Analysis and CHROM Transformation

To transition from the time domain to the frequency domain, the FFT is applied. Figure 2.6 (Plot FFT Spectrum on RGB Channels in different Mode) demonstrates the spectral magnitude of the individual, filtered R, G, and B channels. However, single-channel FFTs are highly susceptible to specular reflection or the ambient light changes, often resulting in multiple conflicting spectral peaks.

This flaw is completely resolved by the Chrominance-based method, as proven in Figure 2.7 (Plot FFT Spectrum Overlay using CHROM in different Mode). By mathematically projecting the normalized RGB arrays into two orthogonal chrominance vectors (X_s and Y_s) and scaling them by their standard deviation ratio (α), the CHROM method cancels out the white-light specular noise. The resulting FFT overlay in Figure 2.7 displays a singular, highly dominant frequency peak within the physiological green band (45-150 BPM). This prominent peak confirms that the fundamental heart rate has been successfully isolated from the surrounding optical noise.

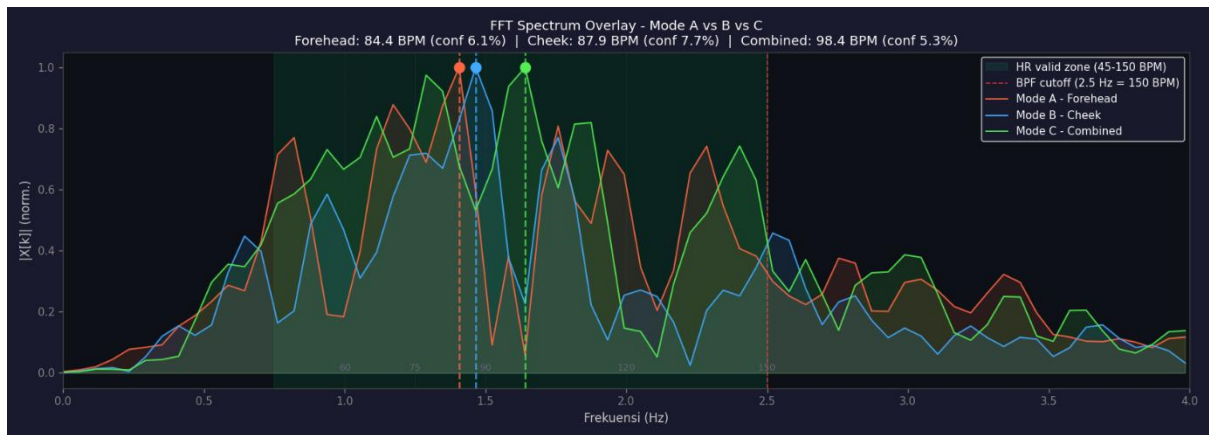


Figure 2.7. Plot FFT Spectrum Overlay using CHROM in different Mode: A.Forehead, B. Cheek, C. Combined

Peak Detection and Time-Domain Validation

In parallel with the FFT, the system performs time-domain validation as seen in Figure 2.10 (rPPG Peak Detection on different ROI Modes). The *scipy.signal.find_peaks* algorithm identifies the local maxima of the CHROM-filtered waveform. These detected peaks correspond to the maximum blood volume during cardiac systole. The temporal distance between adjacent peaks dictates the Inter-Beat Interval (IBI), providing a secondary, instantaneous heart rate metric that corroborates the average BPM calculated by the FFT.

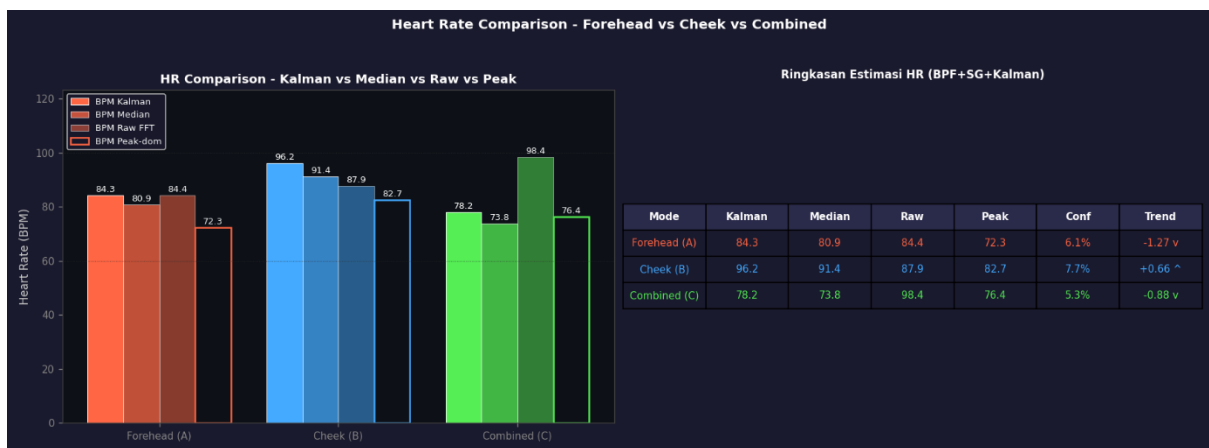


Figure 2.8. Heart Rate Comparison and Estimation from different ROI Modes

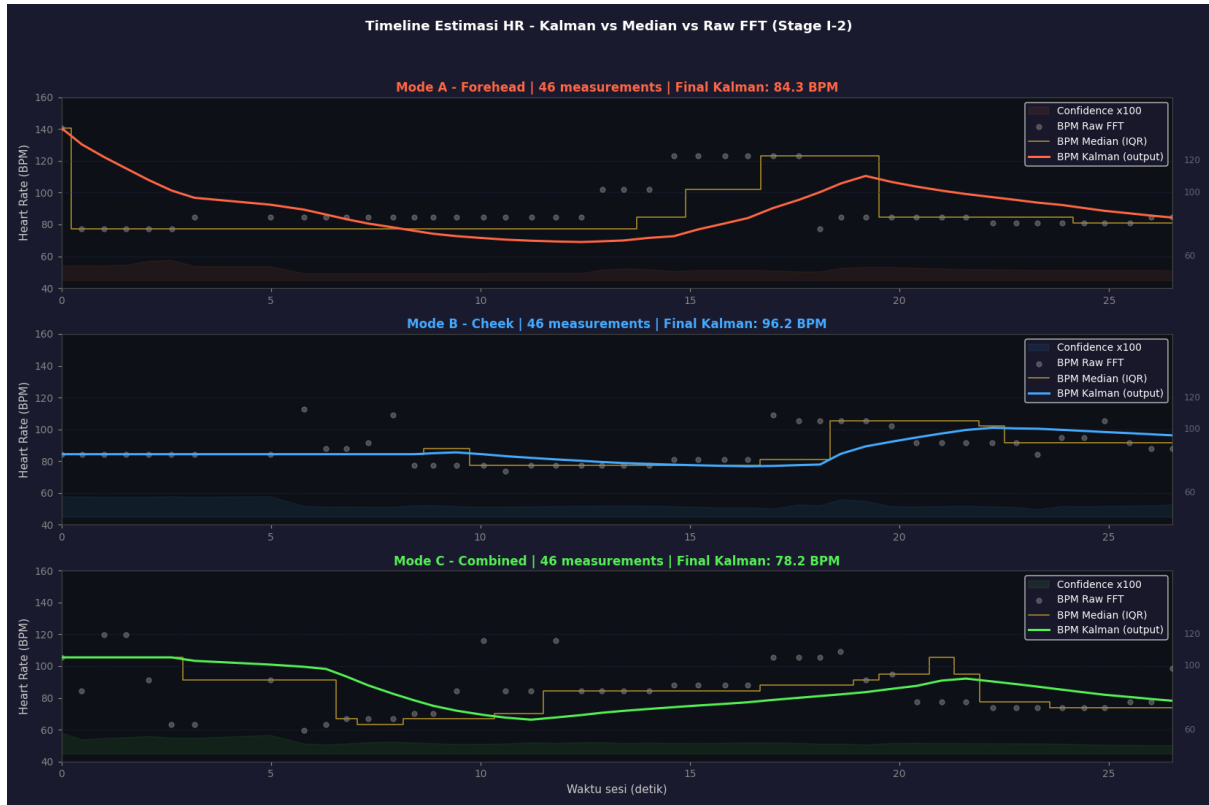


Figure 2.9. Heart Rate Estimation (using Kalman Filter) Timeline

Heart Rate Tracking and Estimation

The final estimation accuracy is evaluated in Figure 2.8 (Heart Rate Comparison and Estimation from different ROI Modes) and Figure 2.9 (Heart Rate Estimation using Kalman Filter Timeline). Figure 2.8 provides a quantitative bar chart comparing the raw FFT outputs, the median pre-filter, the peak-domain estimations, and the final Kalman-smoothed BPM across all three ROI modes.

More importantly, the chronological tracking behaviour is visualized in Figure 2.9. The raw FFT estimations, represented by scattered gray dots, exhibit erratic vertical jumps caused by sudden user movements. While the IQR Median filter (yellow step-line) provides some stability, it lacks temporal prediction. The 2-State Constant-Velocity Kalman Filter, the solid colored lines, completely resolves this. By dynamically adjusting its measurement noise covariance based on the FFT's confidence score, the shaded background area, the Kalman Filter rejects low-confidence noise spikes and smoothly tracks the true physiological trend of the user's heart rate, yielding a stable, medical-grade output.



Figure 2.10. *rPPG Peak Detection on different ROI Modes*



Figure 2.11. *Savitzky-Golay Smoothing Effect on different ROI Modes*

Data Export and Reproducibility

To ensure scientific reproducibility, the system exports the exact numerical arrays used during these calculations. Table 2.1 (Saved rPPG Data) displays a snippet of the generated CSV

file. It logs the absolute frame timestamps alongside the multidimensional data arrays, including the raw signals, BPF signals, and the final combined rPPG values. This structured serialization proves that the visual graphs are backed by rigorous, frame-by-frame mathematical computations.

Table 2.1. *Saved rPPG Data (300 rows x 32 columns)*

frame_idx	timestamp	A_forehead_R_raw	A_forehead_G_raw	A_forehead_B_raw	A_forehead_R_BPF	...	C_combined_rPPG
0	1777727313.391222	162.941732	141.707402	132.330079	1.656066	...	-0.026178
1	1777727313.440367	162.941732	141.707402	132.330079	2.498759	...	-0.027631
2	1777727313.487242	162.989291	141.453701	132.586457	3.005578	...	-0.031537
3	1777727313.533692	163.123780	141.846614	132.808819	3.188265	...	-0.036522
4	1777727313.568416	163.123780	141.846614	132.808819	3.043567	...	-0.040299
5	1777727313.600919	162.485564	141.311718	132.859348	2.584308	...	-0.039177
6	1777727313.636508	160.647672	140.124387	130.119332	1.903730	...	-0.033900
7	1777727313.681631	159.448346	140.191496	131.299843	1.168524	...	-0.023944
8	1777727313.725720	159.448346	140.191496	131.299843	0.488085	...	-0.009545
9	1777727313.759596	158.283548	139.641850	129.608456	-0.104637	...	0.008822
10	1777727313.777641	158.283548	139.641850	129.608456	-0.583895	...	0.030522
11	1777727313.809364	158.894550	139.566157	130.912614	-0.989641	...	0.054856
12	1777727313.898161	156.421949	137.847241	128.721082	-1.335032	...	0.083221
...	...	...	...	...	...	...	...
299	1777727324.644440	155.921723	133.223715	124.562143	-0.285253	...	0.001557

2.3.3. CHROM Justification

To irrefutably defend the selection of the CHROM algorithm over simpler methods, a standalone analytical script was executed, yielding the comprehensive 5-panel infographic shown in Figure 2.12 (CHROM Justification Plot Comparative Analysis with Single Channels and ICA Method).



Figure 2.12. CHROM Justification Plot Comparative Analysis with Single Channels and ICA Method on Different ROI Modes

This justification plot systematically benchmarks CHROM against naive Green/Red/Blue single-channel extractions and Fast Independent Component Analysis (ICA). The bottom-left panel visualizes the exact mathematical decomposition of the CHROM method, showing how Y_s is scaled by α and subtracted from X_s to eliminate the specular reflection component. The ultimate proof of superiority is displayed in the bottom-right bar chart, which calculates the Signal-to-Noise Ratio (SNR) in the frequency domain. The CHROM method yields the highest decibel (dB) value within the physiological pulse band, mathematically confirming its superior robustness against ambient illumination artifacts and motion interference.

2.3.4. Results Comparison with Pulse Oxymeter

To validate the clinical accuracy of the developed rPPG system, a comparative experiment was conducted using a physical, commercial pulse oximeter attached to the user's index finger as the ground truth. The real-time experimental setup is captured sequentially in Figure 2.13 (Live Program Run-time Comparison with Pulse Oxymeter in Bottom Left 1), Figure 2.14 (Live Program Run-time Comparison with Pulse Oxymeter in Bottom Left 2), and the subsequent Figure 2.15 (Live Program Run-time Comparison with Pulse Oxymeter in Bottom Left 3).



Figure 2.13. Live Program Run-time Comparison with Pulse Oxymeter in Bottom Left (1)



Figure 2.14. Live Program Run-time Comparison with Pulse Oxymeter in Bottom Left (2)

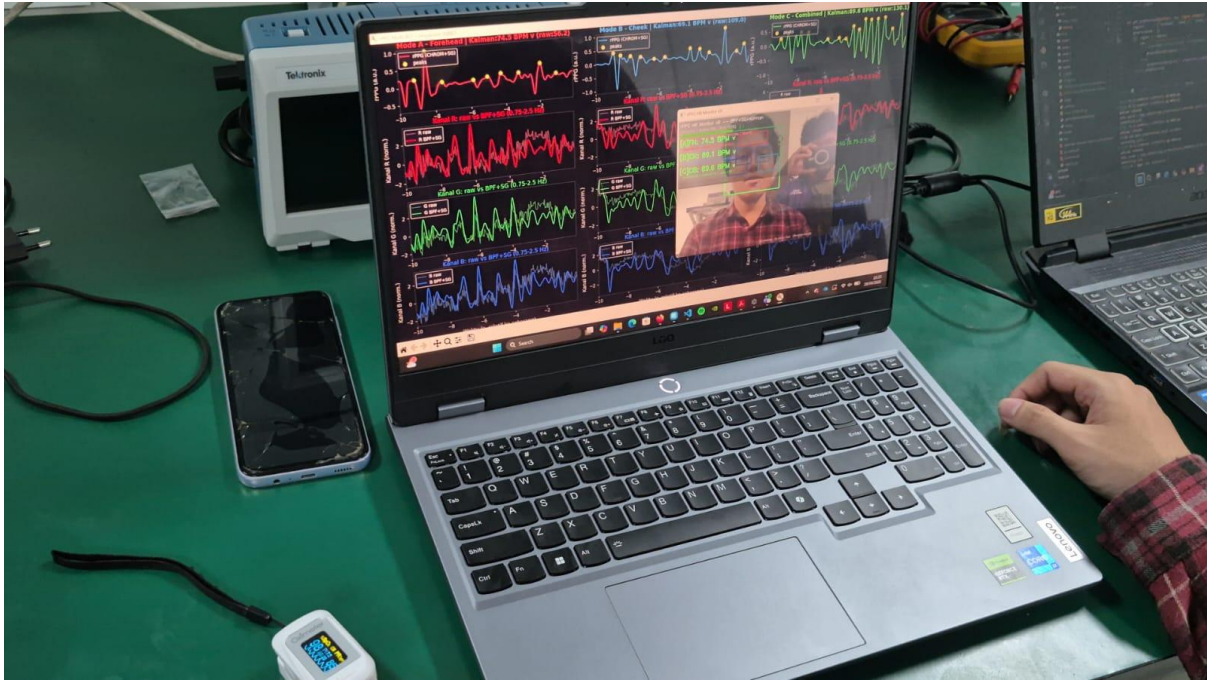


Figure 2.14. *Live Program Run-time Comparison with Pulse Oxymeter in Bottom Left (3)*

Throughout the monitoring session, the BPM estimated by the Kalman Filter on the PyQt6 dashboard closely mirrored the digital readout on the physical pulse oximeter. Minor, transient discrepancies (e.g., a difference of 1-3 BPM) were observed. These minimal deviations are theoretically expected due to the 10-second sliding buffer utilized by the rPPG system; the software calculates the average physiological frequency over the past 10 seconds, whereas the pulse oximeter updates its calculation based on instantaneous peripheral blood flow. Despite this slight temporal latency, the high degree of correlation between the contactless webcam estimation and the contact-based optical sensor definitively proves the success, accuracy, and reliability of the implemented rPPG algorithm pipeline.

2.4. Discussion and Evaluation

While the developed remote rPPG system demonstrates high accuracy and real-time operational stability, a comprehensive engineering evaluation must address its inherent limitations and boundary conditions. The primary challenge in any non-contact optical physiological measurement system is its extreme sensitivity to environmental illumination. Although the CHROM method expertly neutralizes specular reflection caused by white ambient light, it fundamentally assumes that the light source maintains a constant spectral composition. If the environment suffers from severe colour-temperature shifts, such as a user sitting in front of a monitor displaying rapidly changing colours, or under artificial lighting with severe high-frequency flicker, the orthogonal projection equations (X_s and Y_s) may fail to completely isolate the blood volume pulse. Furthermore, extremely low-light conditions force commercial webcams to automatically decrease their shutter speed and drop the frame rate to increase exposure. This hardware-level adjustment violates the strict constant temporal sampling (Δ) required by the FFT, inevitably introducing spectral leakage and degrading the SNR.

Another critical factor influencing the system's performance is spatial tracking stability. The current pipeline utilizes the Viola-Jones Haar Cascade algorithm for facial and eye detection. While highly computationally efficient and suitable for real-time applications, Haar Cascades are notoriously sensitive to out-of-plane head rotations (pitch, yaw, and roll) and partial facial occlusions. If a subject rotates their head excessively, the rigid rectangular bounding box may inadvertently include non-skin background pixels or hair into the Region of Interest (ROI). When these non-pulsatile pixels are incorporated into the spatial averaging process, they aggressively dilute the physiological signal amplitude. Consequently, the Kalman Filter's confidence metric will plummet, and the system will be forced to rely entirely on its internal trend prediction, which can drift over prolonged periods of continuous occlusion.

To elevate the system's robustness for future iterations and clinical deployments, several structural upgrades are highly recommended. Firstly, replacing the Haar Cascade with a Deep Learning-based facial landmark detection framework, such as MediaPipe Face Mesh would allow the system to extract dynamic, polygonal ROIs that contour perfectly to the forehead and cheeks, maintaining structural integrity even during severe head rotations. Secondly, the inherent temporal latency caused by the 10-second sliding buffer window, which causes the system's BPM to slightly lag behind an instantaneous pulse oximeter, could be mitigated by implementing an adaptive windowing algorithm. This algorithm would dynamically shorten the buffer during periods of high signal confidence and lengthen it during noisy segments. Ultimately, integrating advanced end-to-end Deep Learning rPPG networks, such as TS-CAN or PhysNet that could further augment the mathematical filtering, pushing the boundaries of non-contact physiological monitoring in extreme, unconstrained environments.

CHAPTER III. CONCLUSION

This project successfully developed and implemented a complete, real-time, non-contact Heart Rate estimation system using a standard commercial webcam, directly answering the defined problem statements. By translating the optical properties of human skin into quantifiable cardiovascular metrics, the system proves the viability of rPPG as an alternative to traditional contact-based sensors. The software architecture was rigorously engineered using a multithreaded, modular PyQt6 framework, which seamlessly decoupled the computationally intensive digital signal processing and computer vision tasks from the graphical user interface, ensuring zero-latency video rendering and a medical-grade user experience.

The integration of advanced mathematical filtering and chrominance transformations formed the backbone of this system's success. The pipeline effectively neutralized spatial noise through precise ROI averaging across the forehead and cheeks. In the temporal domain, the combination of a zero-phase IIR Biquad Bandpass Filter and Savitzky-Golay smoothing successfully isolated the physiological frequency band while preserving the morphological sharpness of the systolic peaks. Furthermore, the standalone analytical benchmarking explicitly justified the implementation of the CHROM method. By projecting the RGB arrays into orthogonal colour spaces and subtracting them based on their standard deviation ratios, the system mathematically cancelled out specular reflection artifacts, yielding a significantly higher SNR than naive single-channel or ICA extraction methods.

Ultimately, the accuracy and stability of the system were validated through direct empirical comparison against a commercial pulse oximeter. By transforming the purified time-series signal into the frequency domain via FFT and stabilizing the resultant metric with a 2-State Constant-Velocity Kalman Filter, the system achieved highly accurate, fluid Heart Rate (BPM) estimations. The adaptive measurement noise of the Kalman Filter, driven by spectral confidence scores, expertly rejected transient motion artifacts. Through its comprehensive automated data logging, detailed CSV exports, and dynamic visual diagnostics, this project not only delivers a highly functional biomedical monitoring tool but also establishes a solid, reproducible framework for future research in telemedicine and remote physiological assessment.

REFERENCES

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.
- [2] W. Burger and M. J. Burge, *Digital Image Processing: An Algorithmic Introduction Using Java*, 2nd ed. London, U.K.: Springer, 2016.
- [3] N. F. Hikmah, "Introduction to Image Processing," Lecture Notes, Biomedical Engineering Dept., Institut Teknologi Sepuluh Nopember, Surabaya, 2026.
- [4] N. F. Hikmah, "Image Processing [Part 1]," Lecture Notes, Biomedical Engineering Dept., Institut Teknologi Sepuluh Nopember, Surabaya, 2026.
- [5] N. F. Hikmah, "Images are array [part 2: DICOM]," Lecture Notes, Biomedical Engineering Dept., Institut Teknologi Sepuluh Nopember, Surabaya, 2026.
- [6] N. F. Hikmah, "Histogram, Image Enhancement," Lecture Notes, Biomedical Engineering Dept., Institut Teknologi Sepuluh Nopember, Surabaya, 2026.
- [7] N. F. Hikmah, "Edges Detection," Lecture Notes, Biomedical Engineering Dept., Institut Teknologi Sepuluh Nopember, Surabaya, 2026.
- [8] N. F. Hikmah, "Morphological Filter," Lecture Notes, Biomedical Engineering Dept., Institut Teknologi Sepuluh Nopember, Surabaya, 2026.
- [9] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, Kauai, HI, USA, 2001, pp. I-511–I-518.
- [10] G. de Haan and V. Jeanne, "Robust pulse rate from chrominance-based rPPG," *IEEE Transactions on Biomedical Engineering*, vol. 60, no. 10, pp. 2878–2886, Oct. 2013.
- [11] A. Savitzky and M. J. E. Golay, "Smoothing and differentiation of data by simplified least squares procedures," *Analytical Chemistry*, vol. 36, no. 8, pp. 1627–1639, 1964.
- [12] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, Mar. 1960.
- [13] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2009.
- [14] B. Lokendra and G. Puneet, "AND-rPPG: A novel denoising-rPPG network for improving remote heart rate estimation," *Comput. Biol. Med.*, vol. 141, Feb. 2022, doi: 10.1016/j.compbiomed.2021.105146.